

Last updated on **May 11, 2026**

MLflow on Databricks

This article describes how MLflow on Databricks is used to develop high-quality generative AI agents and machine learning models.

NOTE

If you're just getting started with Databricks, consider trying MLflow on [Databricks Free Edition](#).

What is MLflow?

MLflow is the largest open source **AI engineering platform for agents, LLMs, and ML models**. MLflow enables teams of all sizes to debug, evaluate, monitor, and optimize production-quality AI applications while controlling costs and managing access to models and data. With over 30 million monthly downloads, thousands of organizations rely on MLflow each day to ship AI to production with confidence.

MLflow's comprehensive feature set for agents and LLM applications includes production-grade [observability](#), [evaluation](#), [prompt management](#), an [AI Gateway](#) for managing costs and model access, and more.

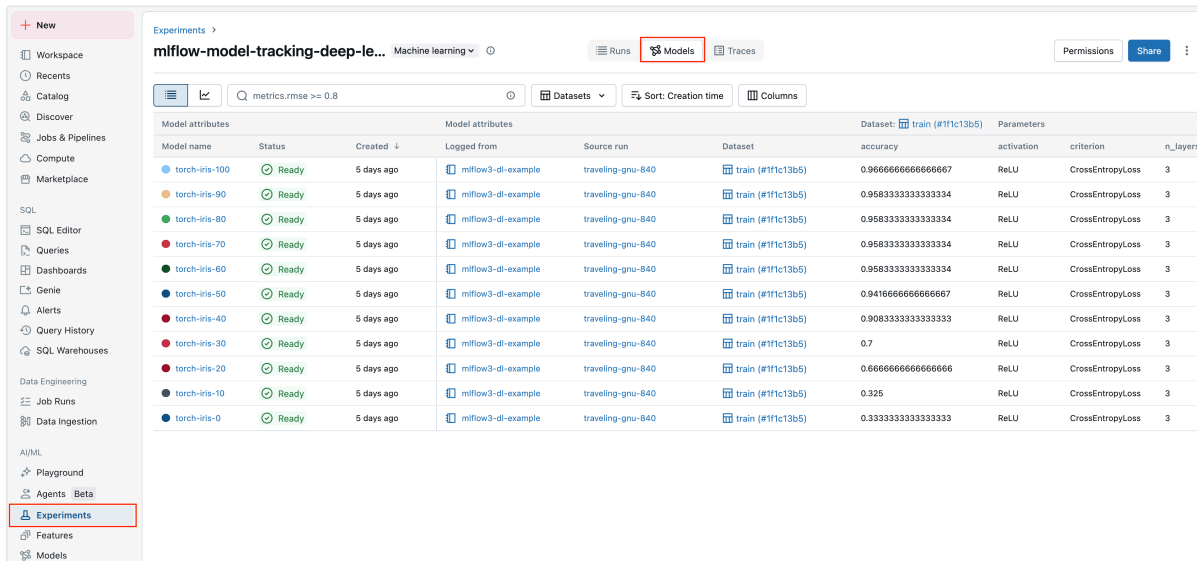
For machine learning (ML) model development, MLflow provides [experiment tracking](#), [model evaluation capabilities](#), a [production model registry](#), and [model deployment tools](#).

MLflow supports any LLM provider, agent framework, ML library, and programming language. MLflow provides native SDKs for [Python](#), [TypeScript/JavaScript](#), [Java](#), and [R](#).

MLflow 3

MLflow 3 on Databricks delivers state-of-the-art observability, evaluation, and prompt management for agents and LLM applications. For ML model development, MLflow 3 provides experiment tracking, model evaluation, a production model registry, and model deployment tools. Using MLflow 3 on Databricks, you can:

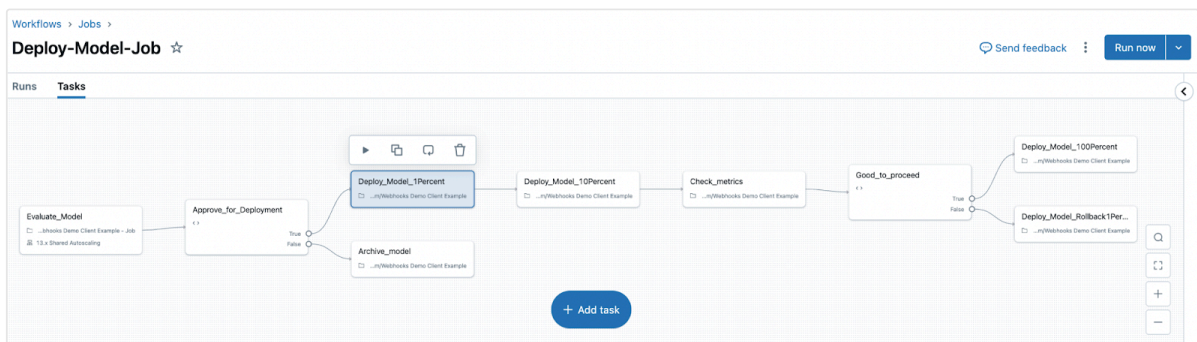
- Centrally track and analyze the performance of your models, AI applications, and agents across all environments, from interactive queries in a development notebook through production batch or real-time serving deployments.



The screenshot shows the MLflow Experiments page for a workspace named 'miflow-model-tracking-deep-le...'. The 'Models' tab is selected. A search filter 'metrics.rmse >= 0.8' is applied. The table below lists various model runs, all with a 'Ready' status and created 5 days ago.

Model name	Status	Created	Logged from	Source run	Dataset	accuracy	activation	criterion	n_layers
torch-iris-100	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.9666666666666667	ReLU	CrossEntropyLoss	3
torch-iris-90	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-80	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-70	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-60	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-50	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.9416666666666667	ReLU	CrossEntropyLoss	3
torch-iris-40	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.9083333333333333	ReLU	CrossEntropyLoss	3
torch-iris-30	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.7	ReLU	CrossEntropyLoss	3
torch-iris-20	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.6666666666666666	ReLU	CrossEntropyLoss	3
torch-iris-10	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.325	ReLU	CrossEntropyLoss	3
torch-iris-0	Ready	5 days ago	miflow3-di-example	traveling-gnu-840	train (#1fc13b5)	0.3333333333333333	ReLU	CrossEntropyLoss	3

- Orchestrate evaluation and deployment workflows using Unity Catalog and access comprehensive status logs for each version of your model, AI application, or agent.



- View and access model metrics and parameters from the model version page in Unity Catalog and from the REST API.

Catalog Explorer > docs > default > tool-calling-agent >

tool-calling-agent version 1

Overview Lineage Traces

Description
Add description

Metrics

Search metrics

Metric	Dataset	Source run	Value
agent/latency_seconds/average	eval_dataset (#229ff317)	traveling-goose-755	2.8506666666666667
response/llm_judged/safety/r...	eval_dataset (#229ff317)	traveling-goose-755	1
response/llm_judged/correct...	eval_dataset (#229ff317)	traveling-goose-755	0.3333333333333333
response/overall_assessment...	eval_dataset (#229ff317)	traveling-goose-755	0.3333333333333333

Parameters

max_tokens	2000
temperature	0.1

Activity log

Time	Log
No log entries	

About this model version

Created by

Created at Mar 31, 2025, 10:56 AM

Last modified Mar 31, 2025, 10:56 AM

Model ID m-c5baf2d6c64a4dc4...

Status Ready

Source run smiling-stag-362

Active serving endpoints
Model version not currently served

Aliases
Add alias

Tags
Add tags

Deployment job
To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created. [Learn more.](#)

Connect deployment job

- Annotate requests and responses (*traces*) for all of your gen AI applications and agents, enabling human experts and automated techniques (such as LLM-as-a-judge) to provide rich feedback. You can leverage this feedback to assess and compare the performance of application versions and to build datasets for improving quality.

Experiments >

Mlflow 3.0 Agent Eval Example

GenAI apps & agents

Traces Evaluations Labeling Versions Monitoring

Permissions Share

Logs Charts Beta Time Range: All

Search traces by request

Filters Sort Columns

Request	Response	Tokens	Execution time	Request time	State
{ "content": "What are user-defined functions (UDFs)?", "additional_kwargs": {}, "re...	{ "content": "Sure, I'd be happy to expla...		2.731s	7 months ago	OK
{ "content": "What is Unity Catalog?", "additional_kwargs": {}, "response_metadata": ...	{ "content": "Unity Catalog is a unified d...		3.186s	7 months ago	OK
{ "content": "What is MLflow Tracking and how does it work?", "additional_kwargs": ...	{ "content": "MLflow Tracking is an ope...		5.522s	7 months ago	OK
{ "question": "What are user-defined functions (UDFs)?" }	{ "content": "User-defined functions (U...		3.690s	7 months ago	OK
{ "question": "What is Unity Catalog?" }	{ "content": "Unity Catalog is a unified d...		4.020s	7 months ago	OK
{ "question": "What is MLflow?" }	{ "content": "MLflow is an open-source ...		4.458s	7 months ago	OK

These capabilities simplify and streamline evaluation, deployment, debugging, and monitoring for all of your AI initiatives.

MLflow 3 also introduces the concepts of Logged Models and Deployment Jobs.

- Logged Models** help you track a model's progress throughout its lifecycle. When you log a model using `log_model()`, a `LoggedModel` is created that persists  Ask Genie

throughout the model's lifecycle, across different environments and runs, and contains links to artifacts such as metadata, metrics, parameters, and the code used to generate the model. You can use the Logged Model to compare models against each other, find the most performant model, and track down information during debugging.

- [Deployment jobs](#) can be used to manage the model lifecycle, including steps like evaluation, approval, and deployment. These model workflows are governed by Unity Catalog, and all events are saved to an activity log that is available on the model version page in Unity Catalog.

See the following articles to install and get started using MLflow 3.

- [Get started with MLflow 3 for models.](#)
- [Track and compare models using MLflow Logged Models.](#)
- [Model Registry improvements with MLflow 3.](#)
- [MLflow 3 deployment jobs.](#)

Databricks-managed MLflow

Databricks provides a fully managed and hosted version of MLflow, building on the open source experience to make it more robust and scalable for enterprise use.

Agents and LLM applications

MLflow on Databricks provides a complete platform for developing, evaluating, and monitoring agents and LLM applications.

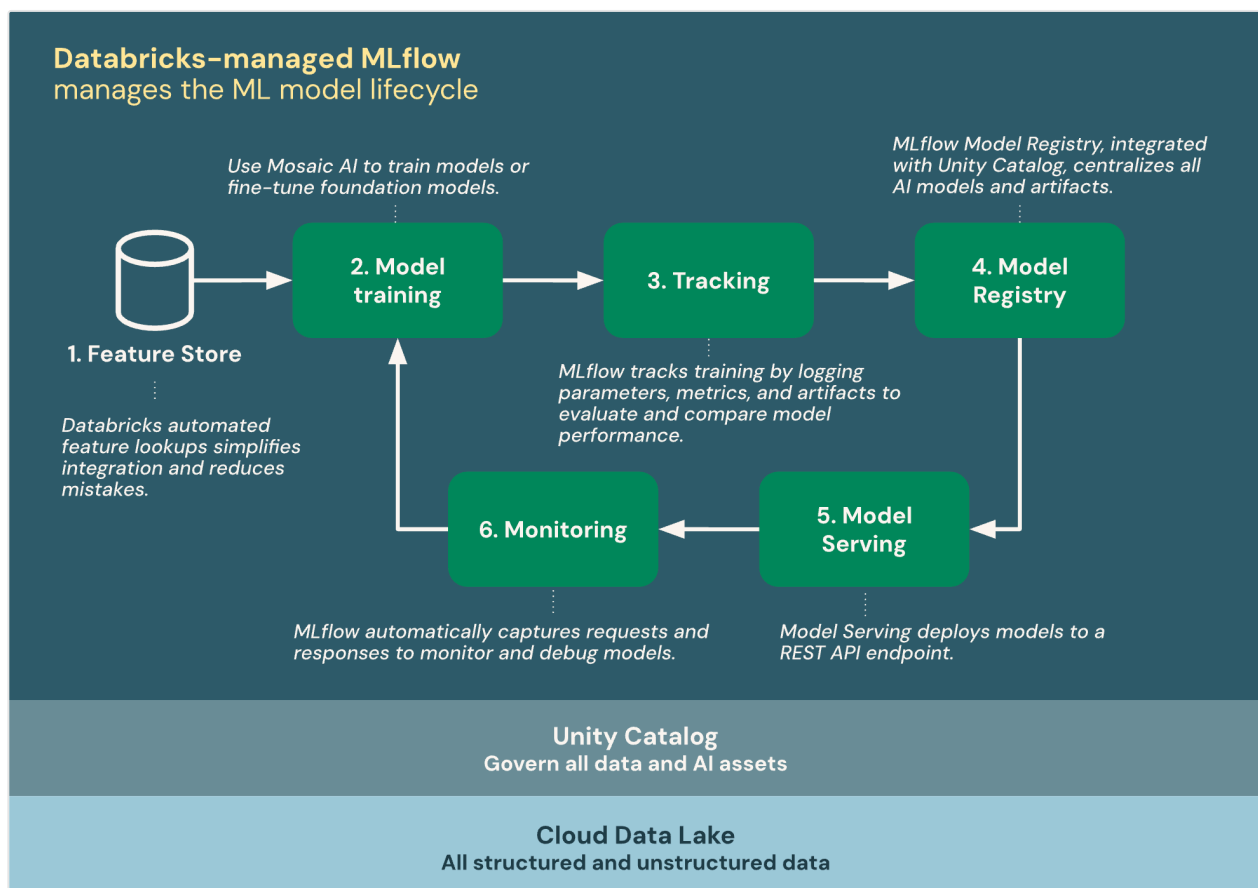
- **Observability:** [MLflow Tracing](#) records the inputs, outputs, and metadata associated with each intermediate step of a request, letting you quickly find the source of unexpected behavior in agents.
- **Evaluation:** Use [Agent Evaluation](#) to measure and improve agent quality, powered by MLflow evaluation.
- **Prompt management:** Version, manage, and iterate on prompt templates used across your AI applications.

- **Agent development:** Use [Agent Framework](#) to create agents, which relies on MLflow to track agent code, performance metrics, and traces.
- **Interactive debugging:** Use [Genie Code for agent observability and evaluation](#) for natural language access to traces, evaluation runs, scorers, and more within your MLflow experiment.

ML model development

MLflow on Databricks provides experiment tracking, model evaluation, a production model registry, and model deployment tools for ML model development.

The following diagram shows how Databricks integrates with MLflow to train and deploy machine learning models.



Databricks-managed MLflow is built on Unity Catalog and the Cloud Data Lake to unify all your data and AI assets in the ML lifecycle:

- 1. Feature Store:** Databricks automated feature lookups simplifies integration and reduces mistakes.

2. **Train models:** Use Databricks AI features to train models or fine-tune foundation models.
3. **Tracking:** MLflow tracks training by logging parameters, metrics, and artifacts to evaluate and compare model performance.
4. **Model Registry:** MLflow Model Registry, integrated with Unity Catalog centralizes AI models and artifacts.
5. **Model Serving:** Model Serving deploys models to a REST API endpoint.
6. **Monitoring:** Model Serving automatically captures requests and responses to monitor and debug models. MLflow augments this data with trace data for each request.

Model training

MLflow Models are at the core of AI and ML development on Databricks. MLflow Models are a standardized format for packaging machine learning models and generative AI agents. The standardized format ensures that models and agents can be used by downstream tools and workflows on Databricks.

- MLflow documentation – [Models](#).

Databricks provides features to help you train different kinds of ML models.

- [Train AI models](#).

Experiment tracking

Databricks uses MLflow experiments as organizational units to track your work while developing models.

Experiment tracking lets you log and manage parameters, metrics, artifacts, and code versions during machine learning training and agent development. Organizing logs into experiments and runs allows you to compare models, analyze performance, and iterate more easily.

- [Experiment tracking using Databricks](#).
- See MLflow documentation for general information on [runs and experiment tracking](#).

Model Registry with Unity Catalog

MLflow Model Registry is a centralized model repository, UI, and set of APIs for managing the model deployment process.

Databricks integrates Model Registry with Unity Catalog to provide centralized governance for models. Unity Catalog integration allows you to access models across workspaces, track model lineage, and discover models for reuse.

- [Manage models using Databricks Unity Catalog.](#)
- See MLflow documentation for general information on [Model Registry](#).

Model Serving

Databricks Model Serving is tightly integrated with MLflow Model Registry and provides a unified, scalable interface for deploying, governing, and querying AI models. Each model you serve is available as a REST API that you can integrate into web or client applications.

While they are distinct components, Model Serving heavily relies on MLflow Model Registry to handle model versioning, dependency management, validation, and governance.

- [Model Serving using Databricks.](#)

Open source vs. Databricks-managed MLflow features

For general MLflow concepts, APIs, and features shared between open source and Databricks-managed versions, refer to [MLflow documentation](#). For features exclusive to Databricks-managed MLflow, see Databricks documentation.

The following table highlights the key differences between open source MLflow and Databricks-managed MLflow and provides documentation links to help you learn more:

Feature	Availability on open source MLflow	Availability on Databricks-managed MLflow
Security	User must provide their own security governance layer	Databricks enterprise-grade security
Disaster recovery	Unavailable	Databricks disaster recovery
Experiment tracking	MLflow Tracking API	MLflow Tracking API integrated with Databricks advanced experiment tracking
Model Registry	MLflow Model Registry	MLflow Model Registry integrated with Databricks Unity Catalog
Unity Catalog integration	Open source integration with Unity Catalog	Databricks Unity Catalog
Model deployment	User-configured integrations with external serving solutions (SageMaker, Kubernetes, container services, and so on)	Databricks Model Serving and external serving solutions
AI agents	MLflow LLM development	MLflow LLM development integrated with Agent Framework and Agent Evaluation
Encryption	Unavailable	Encryption using customer-managed keys

Open source telemetry collection was introduced in MLflow 3.2.0, and is **disabled on Databricks by default**. For more details, refer to the [MLflow usage tracking documentation](#).

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **May 4, 2026**

Get started with MLflow 3 for models

📘 NOTE

This article focuses on MLflow 3 features for traditional machine learning and deep learning models. MLflow 3 also offers comprehensive features for GenAI application development including tracing, evaluation, and human feedback collection. See [MLflow 3 for GenAI](#) for details.

This article gets you started with MLflow 3 for developing machine learning models. It describes how to install MLflow 3 for models and includes several demo notebooks to get started. It also includes links to pages that cover the new features of MLflow 3 for models in more detail.

What is MLflow 3 for models?

MLflow 3 for models on Databricks delivers state-of-the-art experiment tracking, performance evaluation, and production management for machine learning models. MLflow 3 introduces significant new capabilities while preserving core tracking concepts, making migration from MLflow 2.x quick and simple.

What is MLflow 3 for GenAI?

Beyond MLflow 3 for Models, MLflow 3 for GenAI introduces a wide variety of new features and improvements for agent and GenAI application development. For a comprehensive overview, see [MLflow 3 for GenAI](#).

Core features in MLflow 3 for GenAI include:

 Ask Genie

- **Tracing and observability** – End-to-end observability for GenAI applications with automatic instrumentation for 20+ frameworks including OpenAI, LangChain, LlamaIndex, and Anthropic
- **Evaluation and monitoring** – Comprehensive GenAI evaluation capabilities to measure and improve quality from development through production. Includes built-in LLM judges, customizable judges, evaluation dataset management, and real-time monitoring.
- **Human feedback collection** – Customizable Review UI for collecting domain expert feedback and interactively testing agents, with structured **labeling sessions** for organizing and tracking review progress
- **Prompt Registry** – Centralized prompt versioning, management, and A/B testing with Unity Catalog integration

How is MLflow 3 for models different from MLflow 2

MLflow 3 for models on Databricks enables you to:

- Centrally track and analyze the performance of your models across all environments, from interactive queries in a development notebook through production batch or real-time serving deployments.

Model attributes			Model attributes			Dataset: train (#1f1c13b5)		Parameters	
Model name	Status	Created	Logged from	Source run	Dataset	accuracy	activation	criterion	n_layers
torch-iris-100	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9666666666666667	ReLU	CrossEntropyLoss	3
torch-iris-90	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-80	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-70	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-60	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-50	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9416666666666667	ReLU	CrossEntropyLoss	3
torch-iris-40	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9083333333333333	ReLU	CrossEntropyLoss	3
torch-iris-30	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.7	ReLU	CrossEntropyLoss	3
torch-iris-20	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.6666666666666666	ReLU	CrossEntropyLoss	3
torch-iris-10	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.325	ReLU	CrossEntropyLoss	3
torch-iris-0	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.3333333333333333	ReLU	CrossEntropyLoss	3

- View and access model metrics and parameters from the model version page in Unity Catalog and from the REST API, across all workspaces and experiments.

Catalog Explorer > docs > default > tool-calling-agent >

tool-calling-agent version 1

Copy this version

Overview Lineage Traces

Description
Add description

Metrics

Search metrics

Metric	Dataset	Source run	Value
agent/latency_seconds/average	eval_dataset (#229ff317)	traveling-goose-755	2.8506666666666667
response/llm_judged/safety/r...	eval_dataset (#229ff317)	traveling-goose-755	1
response/llm_judged/correct...	eval_dataset (#229ff317)	traveling-goose-755	0.3333333333333333
response/overall_assessment...	eval_dataset (#229ff317)	traveling-goose-755	0.3333333333333333

Parameters

max_tokens	2000
temperature	0.1

Activity log

Time	Log
No log entries	

About this model version

Created by

Created at Mar 31, 2025, 10:56 AM

Last modified Mar 31, 2025, 10:56 AM

Model ID m-c5baf2d6c64a4dc4...

Status Ready

Source run [smiling-stag-362](#)

Active serving endpoints
Model version not currently served

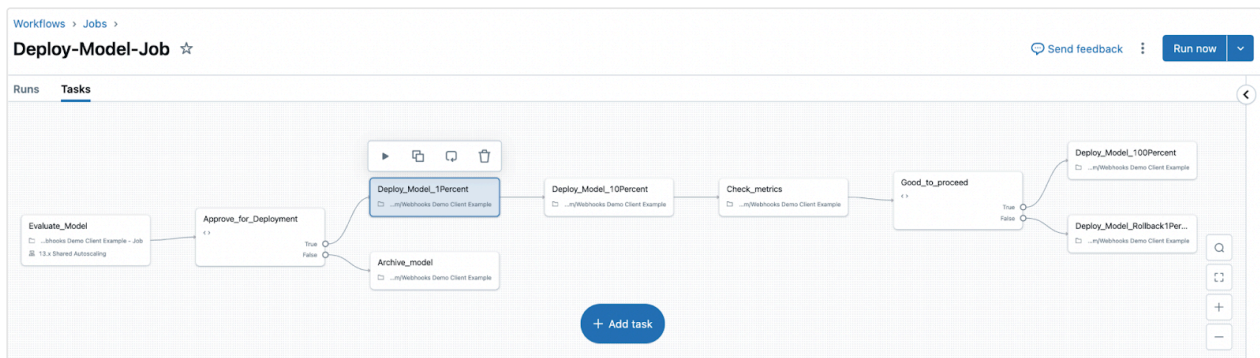
Aliases
Add alias

Tags
Add tags

Deployment job
To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created. [Learn more.](#)

Connect deployment job

- Orchestrate evaluation and deployment workflows using Unity Catalog and access comprehensive status logs for each version of your model.



These capabilities simplify and streamline machine learning model development, evaluation, and production deployment.

Logged Models

Much of the new functionality of MLflow 3 derives from the new concept of a `LoggedModel`. For deep learning and traditional machine learning models, `LoggedModels` elevates the concept of a model produced by a training run, establishing it as a dedicated object to track the model lifecycle across different training and evaluation runs.

`LoggedModels` capture metrics, parameters, and traces across phases of development (training and evaluation) and across environments (development, staging, and production). When a `LoggedModel` is promoted to Unity Catalog as a Model Version, all performance data from the original `LoggedModel` becomes visible on the UC Model Version page, providing visibility across all workspaces and experiments. For more details, see [Track and compare models using MLflow Logged Models](#).

Deployment jobs

MLflow 3 also introduces the concept of a deployment job. Deployment jobs use Lakeflow Jobs to manage the model lifecycle, including steps like evaluation, approval, and deployment. These model workflows are governed by Unity Catalog, and all events are saved to an activity log that is available on the model version page in Unity Catalog.

Migrating from MLflow 2.x

Although there are many new features in MLflow 3, the core concepts of experiments and runs, along with their metadata such as parameters, tags, and metrics, all remain the same. Migration from MLflow 2.x to 3.0 is very straightforward and should require minimal code changes in most cases. This section highlights some key differences from MLflow 2.x and what you should be aware of for a seamless transition.

Logging Models

When logging models in 2.x, the `artifact_path` parameter is used.

```
with mlflow.start_run():
    mlflow.pyfunc.log_model(
        artifact_path="model",
        python_model=python_model,
        ...
    )
```

In MLflow 3, use `name` instead, which allows the model to later be searched by name. The `artifact_path` parameter is still supported but has been deprecated. Additionally, MLflow no longer requires a run to be active when logging a model, because models:

have become first-class citizens in MLflow 3. You can directly log a model without first starting a run.

```
mlflow.pyfunc.log_model(  
    name="model",  
    python_model=python_model,  
    ...  
)
```

Model artifacts

In MLflow 2.x, model artifacts are stored as run artifacts under the run's artifact path. In MLflow 3, model artifacts are now stored in a different location, under the model's artifact path instead.

```
# MLflow 2.x  
experiments/  
└─ <experiment_id>/  
    └─ <run_id>/  
        └─ artifacts/  
            └─ ... # model artifacts are stored here  
  
# MLflow 3  
experiments/  
└─ <experiment_id>/  
    └─ models/  
        └─ <model_id>/  
            └─ artifacts/  
                └─ ... # model artifacts are stored here
```

It is recommended to load models with `mlflow.<model-flavor>.load_model` using the model URI returned by `mlflow.<model-flavor>.log_model` to avoid any issues. This model URI is of the format `models:/<model_id>` (rather than `runs:/<run_id>/<artifact_path>` as in MLflow 2.x) and can also be constructed manually if only the model ID is available.

Model registry

In MLflow 3, the default registry URI is now `databricks-uc`, meaning the MLflow Model Registry in Unity Catalog will be used (see [Manage model lifecycle in Unity Catalog](#) for more details). The names of models registered in Unity Catalog are of the form

`<catalog>.<schema>.<model>`. When calling APIs that require a registered model name, such as `mlflow.register_model`, this full, three-level name is used.

For workspaces that have Unity Catalog enabled and whose [default catalog](#) is in Unity Catalog, you can also use `<model>` as the name and the default catalog and schema will be inferred (no change in behavior from MLflow 2.x). If your workspace has Unity Catalog enabled but its [default catalog](#) is not configured to be in Unity Catalog, you will need to specify the full three-level name.

Databricks recommends using the MLflow Model Registry in Unity Catalog for managing the lifecycle of your models.

If you want to continue using the [Workspace Model Registry \(legacy\)](#), use one of the following methods to set the registry URI to `databricks`:

- Use `mlflow.set_registry_uri("databricks")`.
- Set the environment variable `MLFLOW_REGISTRY_URI`.
- To set the environment variable for registry URI at scale, you can use [init scripts](#). This requires [all-purpose compute](#).

Other important changes

- MLflow 3 clients can load all runs, models, and traces logged with MLflow 2.x clients. However, the reverse is not necessarily true, so models and traces logged with MLflow 3 clients may not be able to be loaded with older 2.x client versions.
- The `mlflow.evaluate` API has been deprecated. For traditional ML or deep learning models, use `mlflow.models.evaluate` which maintains full compatibility with the original `mlflow.evaluate` API. For LLMs or GenAI applications, use the `mlflow.genai.evaluate` API instead.
- The `run_uuid` attribute has been removed from the `RunInfo` object. Use `run_id` instead in your code.

Install MLflow 3

To use MLflow 3, you must update the package to use the correct (≥ 3.0) version. The following lines of code must be executed each time a notebook is run:  Ask Genie

Python

```
%pip install mlflow>=3.0 --upgrade  
dbutils.library.restartPython()
```

Example notebooks

The following pages illustrate the MLflow 3 model tracking workflow for traditional ML and deep learning. Each page includes an example notebook.

- [MLflow 3 traditional ML workflow.](#)
- [MLflow 3 deep learning workflow.](#)

Limitation

While Spark model logging (`mlflow.spark.log_model`) continues to work in MLflow 3, it does not use the new `LoggedModel` concept. Models logged using Spark model logging continue to use MLflow 2.x runs and run artifacts.

Next steps

To learn more about the new features of MLflow 3, see the following articles:

- [Track and compare models using MLflow Logged Models.](#)
- [Model Registry improvements with MLflow 3.](#)
- [MLflow 3 deployment jobs.](#)

Is this page

as this page helpful?

 Yes

 No

 Ask Genie

Last updated on **Feb 25, 2026**

MLflow 3 traditional ML workflow

Example notebook

The example notebook runs a model training job, which is tracked as an MLflow Run, to produce a trained model, which is tracked as an [MLflow Logged Model](#).

MLflow 3 traditional ML model notebook

[Open notebook in new tab](#)

 Copy link for import

MLflow 3.0 traditional ML example

This notebook first runs a model training job, which is tracked as an MLflow Run, to produce a trained model. The model is tracked as an MLflow Logged Model.

```
%pip install mlflow>=3.0 --upgrade
dbutils.library.restartPython()
```

Expand notebook ▼

Explore model parameters and performance using the MLflow UI

To explore the model in the MLflow UI:

 Ask Genie

1. Click **Experiments** in the workspace sidebar.
2. Find your experiment in the experiments list. You can select the **Only my experiments** checkbox or use the **Filter experiments** search box to filter the list of experiments.
3. Click the name of your experiment. The **Runs** page opens. The experiment contains two MLflow runs, one used to train the model and one used to evaluate the model.

Experiments > MLflow 3.0 Traditional ML Exam... Machine learning

Runs Models Traces Permissions Share

metrics.rmse < 1 and params.model = "tree" Time created State: Active Datasets Sort: Created Columns + New run

Group by

Run Name	Created	Dataset	Duration	Source	Models	mae	r2	rmse
skillful-snake-643	5 days ago	test (cc28e2f0)	2.2s	mlflow3...	-	0.3256178...	0.42114262...	0.1344235...
righteous-eel-476	5 days ago	train (d95ce242)	9.3s	mlflow3...	main.default.ml_mo... +1	0.3209158...	0.3953727...	0.1326400...

4. Click the **Models** tab. The `LoggedModel (elasticnet)` is tracked on this screen. You can see all of the parameters and metadata, as well as all of the metrics linked from the training and evaluation runs.

Experiments > MLflow 3.0 Traditional ML Exam... Machine learning

Runs Models Traces Permissions Share

metrics.rmse >= 0.8 Datasets Sort: Creation time Columns

Model name	Status	Created	Logged from	Source run	Registered models	Dataset	rmse	r2
elasticnet	Ready	5 days ago	mlflow3-ml-example	52b6da39d762401eac428d5bcd	main.default.ml_model v3	train (#d95ce242) +1	0.13442354610166166	0.421142624442

5. Click the model name to display the model page, which contains information like the model's parameters and metrics, as well as details such as its source run, relevant datasets, and model versions registered in Unity Catalog.

Experiments > /Users/ > /MLflow 3.0 Traditional ML Example > Models >

elasticnet

Register model

Overview
Traces
Artifacts

Description

No description

Details

Created at	1 minute ago
Status	Ready
Model ID	m-4c8d53d0b3fe4ebabb03a10d7a7c03cc
Source run	righteous-eel-476
Source run ID	52b6da39d762401eac428d5bcd3c121c
Logged from	mlflow3-ml-example
Datasets used	train (#d95ce242) +1
Model versions	main.default.ml_model v3

Metrics (6)

Search metrics

Metric	Dataset	Source run	Value
rmse	train (#d95ce242)	righteous-eel-476	0.132640093646...
r2	train (#d95ce242)	righteous-eel-476	0.395372792495...
mae	train (#d95ce242)	righteous-eel-476	0.3209158276037...
rmse	test (#cc28e2f0)	skillful-snake-643	0.1344235461016...
r2	test (#cc28e2f0)	skillful-snake-643	0.4211426244426...

Parameters (2)

Search parameters

Parameter	Value
alpha	0.5
l1_ratio	0.5

6. The notebook registers the model to Unity Catalog. As a result, all model parameters and performance data are available on the model version page in Catalog Explorer. You can get to this page directly by clicking on the model version from the MLflow model page. Clicking on the model ID and source run here will bring you back to the MLflow model and run pages respectively.

Catalog Explorer > main > default > iris_model >

iris_model version 1

Copy this version

Overview
Lineage
Artifacts
Traces

Description

Add description

Metrics

Search metrics

Metric	Dataset	Source run	Value
rmse	train (#e70179ff)	placid-sheep-690	0.1326400936461997
r2	train (#e70179ff)	placid-sheep-690	0.3953727924959558
mae	train (#e70179ff)	placid-sheep-690	0.3209158276037644
rmse	test (#484d642b)	rumbling-wolf-893	0.13442354610166166
r2	test (#484d642b)	rumbling-wolf-893	0.4211426244426051

Parameters

alpha	0.5
l1_ratio	0.5

Activity log

Time	Log
2025-06-06 13:42:48	Model version 1 registered by user

About this model version

Created by
Created at Jun 06, 2025, 01:42 PM
Last modified Jun 06, 2025, 01:42 PM
Model ID m-e9f28cf1aef540db...
Status Ready
Source run placid-sheep-690

Active serving endpoints

Model version not currently served

Aliases

Add alias

Tags

Add tags

Deployment job

To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created.

Connect deployment job

What's the difference between the **Models** tab on the MLflow experiment page and the model version page in Catalog Explorer?

The **Models** tab of the [experiment page](#) and the model version page in [Catalog Explorer](#) show similar information about the model. The two views have different roles in the model development and deployment lifecycle.

- The **Models** tab of the experiment page presents the results of logged models from an experiment on a single page. The [Charts tab](#) on this page provides visualizations to help you compare models and select the model versions to register to Unity Catalog for possible deployment.
- In Catalog Explorer, the model version page provides an overview of all model performance and evaluation results. This page shows model parameters, metrics, and traces across all linked environments including different workspaces, endpoints, and experiments. This is useful for monitoring and deployment, and works especially well with [deployment jobs](#). The evaluation task in a deployment job creates additional metrics that appear on this page. The approver for the job can then review this page to assess whether to approve the model version for deployment.

Next steps

To learn more about `LoggedModel` tracking introduced in MLflow 3, see the following article:

- [Track and compare models using MLflow Logged Models.](#)

To learn more about using MLflow 3 with deep learning workflows, see the following article:

- [MLflow 3 deep learning workflow.](#)

Is this page

Was this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Feb 25, 2026**

MLflow 3 deep learning workflow

Example notebook

The example notebook runs a single deep learning model training job with PyTorch, which is tracked as an MLflow run. It logs a checkpoint model after every 10 epochs. Each checkpoint is tracked as an MLflow `LoggedModel`. Using MLflow's UI or search API, you can inspect the checkpoint models and rank them by accuracy.

The notebook installs the `scikit-learn` and `torch` libraries.

MLflow 3 deep learning model with checkpoints notebook

[Open notebook in new tab](#)

 Copy link for import

MLflow 3.0 deep learning example

This simple notebook introduces MLflow on Databricks (<https://docs.databricks.com/aws/en/mlflow>) for deep learning use cases. With simple instrumentation using MLflow, you can track metadata, checkpoints, and metrics during your deep learning training jobs.

This notebook uses PyTorch to train a simple classification model, which is tracked as an MLflow Run. It stores a model checkpoint every 10 epochs, along with accuracy metrics. Each checkpoint is tracked as an MLflow `LoggedModel`.

[Expand notebook](#) point is then selected and used for batch inference.

Use the UI to explore model performance and register a model

After running the notebook, you can view the saved checkpoint models in the MLflow experiments UI. A link to the experiment appears in the notebook cell output, or follow these steps:

1. Click **Experiments** in the workspace sidebar.
2. Find your experiment in the experiments list. You can select the **Only my experiments** checkbox or use the **Filter experiments** search box to filter the list of experiments.
3. Click the name of your experiment. The **Runs** page opens. The experiment contains one MLflow run.

Run Name	Created	Dataset	Duration	Source	Models	Metrics
loud-gnat-98	5 days ago	train (1f1c13b5)	1.7min	mlflow3...	main.default.dl_m...	accuracy: 0.9833333...

4. Click the **Models** tab. The individual checkpoint models are tracked on this screen. For each checkpoint, you can see the model's accuracy, along with all of its parameters and metadata.

Model name	Status	Created	Logged from	Source run	Dataset	accuracy	activation	criterion	n_layers
torch-iris-100	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.9833333333333333	ReLU	CrossEntropyLoss	3
torch-iris-90	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.975	ReLU	CrossEntropyLoss	3
torch-iris-80	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.975	ReLU	CrossEntropyLoss	3
torch-iris-70	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.975	ReLU	CrossEntropyLoss	3
torch-iris-60	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.9666666666666667	ReLU	CrossEntropyLoss	3
torch-iris-50	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.9666666666666667	ReLU	CrossEntropyLoss	3
torch-iris-40	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-30	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3
torch-iris-20	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.95	ReLU	CrossEntropyLoss	3
torch-iris-10	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.6583333333333333	ReLU	CrossEntropyLoss	3
torch-iris-0	Ready	5 days ago	mlflow3-dl-example	loud-gnat-98	train (#1f1c13b5)	0.325	ReLU	CrossEntropyLoss	3

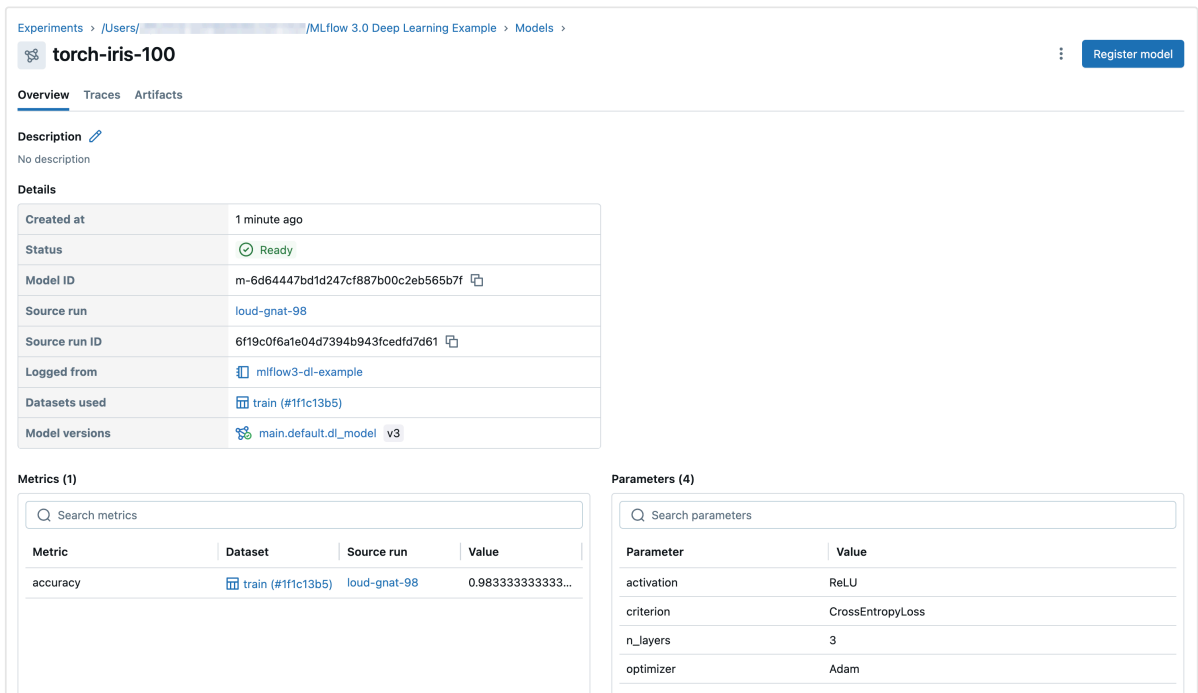
In the example notebook, you registered the best performing model to Unity Catalog. You can also register a model from the UI. To do so, follow these steps:



1. From the **Models** tab, click the name of the model to register.
2. From the model details page, in the upper-right corner, click **Register model**.

 TIP

It can take a few minutes for a model to appear in the UI after registering it. Do not press the **Register model** more than once, otherwise you will register duplicate models.



The screenshot shows the MLflow model details page for a model named 'torch-iris-100'. The page is divided into several sections: Overview, Traces, and Artifacts. The Overview section is active and contains a Description, Details, Metrics, and Parameters.

Details

Created at	1 minute ago
Status	Ready
Model ID	m-6d64447bd1d247cf887b00c2eb565b7f
Source run	loud-gnat-98
Source run ID	6f19c0f6a1e04d7394b943fcedfd7d61
Logged from	mlflow3-dl-example
Datasets used	train (#1f1c13b5)
Model versions	main.default.dl_model v3

Metrics (1)

Metric	Dataset	Source run	Value
accuracy	train (#1f1c13b5)	loud-gnat-98	0.983333333333...

Parameters (4)

Parameter	Value
activation	ReLU
criterion	CrossEntropyLoss
n_layers	3
optimizer	Adam

3. Select **Unity Catalog** and either select an existing model name from the drop-down menu or type in a new name.

Register model

☒ Unity Catalog
 ☐ Workspace Model Registry

Select an existing model or enter a model name to create it

model_name

main.user_docs.model_name

dl_model

main.default.dl_model

4. Click **Register**.

Use the API to rank checkpoint models

The following code shows how to rank the checkpoint models by accuracy. For more details on searching logged models using the API, see [Search and filter Logged Models](#).

Python

```
ranked_checkpoints = mlflow.search_logged_models(
    output_format="list",
    order_by=[{"field_name": "metrics.accuracy", "ascending": False}]
)

best_checkpoint: mlflow.entities.LoggedModel = ranked_checkpoints[0]
print(best_checkpoint.metrics[0])

<Metric:
  dataset_digest='9951783d',
  dataset_name='train',
  key='accuracy',
  model_id='m-bba8fa52b6a6499281c43ef17fcdac84',
  run_id='394928abe6fc4787aaf4e666ac89dc8a',
  step=90,
  timestamp=1730828771880,
  value=0.9553571428571429
```

```
>
worst_checkpoint: mlflow.entities.LoggedModel = ranked_checkpoints[-1]
print(worst_checkpoint.metrics[0])

<Metric:
  dataset_digest='9951783d',
  dataset_name='train',
  key='accuracy',
  model_id='m-88885bc26de7492f908069cfe15a1499',
  run_id='394928abe6fc4787aaf4e666ac89dc8a',
  step=0,
  timestamp=1730828730040,
  value=0.35714285714285715
```

What's the difference between the **Models** tab on the MLflow experiment page and the model version page in Catalog Explorer?

The **Models** tab of the [experiment page](#) and the model version page in [Catalog Explorer](#) show similar information about the model. The two views have different roles in the model development and deployment lifecycle.

- The **Models** tab of the experiment page presents the results of logged models from an experiment on a single page. The [Charts tab](#) on this page provides visualizations to help you compare models and select the model versions to register to Unity Catalog for possible deployment.
- In Catalog Explorer, the model version page provides an overview of all model performance and evaluation results. This page shows model parameters, metrics, and traces across all linked environments including different workspaces, endpoints, and experiments. This is useful for monitoring and deployment, and works especially well with [deployment jobs](#). The evaluation task in a deployment job creates additional metrics that appear on this page. The approver for the job can then review this page to assess whether to approve the model version for deployment.

Next steps

To learn more about `LoggedModel` tracking introduced in MLflow 3, see the following article:

- [Track and compare models using MLflow Logged Models.](#)

To learn more about using MLflow 3 with traditional ML workflows, see the following article:

- [MLflow 3 traditional ML workflow.](#)

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Feb 25, 2026**

Model Registry improvements with MLflow 3

This page describes improvements to the Model Registry in MLflow 3.

In MLflow 3, the Unity Catalog model registry has been enhanced to make it easier to find model parameters and performance data. When you register a `LoggedModel` to the Unity Catalog model registry, all of its metrics and parameters are available in the model registry UI and from the API. You can see model performance metrics across all MLflow experiments and workspaces on a single page. For more information about `LoggedModel`, see [Track and compare models using MLflow Logged Models](#).

Catalog Explorer > main > default > iris_model >

iris_model version 1

Copy this version

Overview

Lineage

Artifacts

Traces

Description

Add description

Metrics

Search metrics

Metric	Dataset	Source run	Value
rmse	train (#e70179ff)	placid-sheep-690	0.1326400936461997
r2	train (#e70179ff)	placid-sheep-690	0.3953727924959558
mae	train (#e70179ff)	placid-sheep-690	0.3209158276037644
rmse	test (#484d642b)	rumbling-wolf-893	0.13442354610166166
r2	test (#484d642b)	rumbling-wolf-893	0.4211426244426051

Parameters

alpha	0.5
l1_ratio	0.5

Activity log

Time	Log
2025-06-06 13:42:48	Model version 1 registered by user

About this model version

Created by

Created at

Last modified

Model ID

Status

Source run

Active serving endpoints

Model version not currently served

Aliases

Add alias

Tags

Add tags

Deployment job

To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created. [Learn more.](#)

Connect deployment job

From the **Traces** panel, you can see traces associated with the Model ID from development and evaluation (`mlflow.evaluate()` runs) alongside traces from online serving in endpoints. You can use the search box to filter traces, as well as click on a trace to see the full set of spans.

Catalog Explorer > docs > default > tool-calling-agent >

tool-calling-agent version 1

Copy this version

Overview Lineage **Traces**

Search traces

Columns

Request ID	Trace name	Time created	Status	Source	Request	Response	Execution time	Tags
tr-535940535c974b799d4...	LangGraph	4 hours ago	OK	traveling-geese-755	> {"messages": [{"role": "user", "content": "What are user-defined functions..."}]}	> {"messages": [{"role": "user", "content": "What are user-defined functions..."}]}	2.572 s	eval.requestId: a122c36b-834e...
tr-c630665088c34d1db3c...	LangGraph	4 hours ago	OK	traveling-geese-755	> {"messages": [{"role": "user", "content": "What is Unity Catalog?"}]}	> {"messages": [{"role": "user", "content": "What is Unity Catalog?", "id": "..."}]}	1.314 s	eval.requestId: ff8e2b58-d6ce-...
tr-89440f605d614073af53...	LangGraph	4 hours ago	OK	traveling-geese-755	> {"messages": [{"role": "user", "content": "What is MLflow Tracking and how..."}]}	> {"messages": [{"role": "user", "content": "What is MLflow Tracking and how..."}]}	5.106 s	eval.requestId: 39a303dd-7c65...

Next steps

To learn more about other new features of MLflow 3, see the following articles:

- [Track and compare models using MLflow Logged Models.](#)
- [MLflow 3 deployment jobs.](#)

Is this page

as this page helpful?

☒ Yes

☐ No

Send feedback

Organize training runs with MLflow experiments

Last updated on **May 8, 2026**

Organize training runs with MLflow experiments

Experiments are units of organization for your MLflow runs, including agent traces, LLM application evaluations, and [model training runs](#). There are two types of experiments: workspace and notebook.

- You can create a workspace experiment from the Databricks UI or the MLflow API. Workspace experiments are not associated with any notebook, and any notebook can log a run to these experiments by using the experiment ID or the experiment name.
- A notebook experiment is associated with a specific notebook. Databricks automatically creates a notebook experiment if there is no active experiment when you start a run using `mlflow.start_run()`.

To see all of the experiments in a workspace that you have access to, in the sidebar, under **AI/ML**, click **Experiments**.

Experiments Compare (0)

Create model training

GenAI apps & agents
Build a Generative AI application or agent
Create experiment

Foundation Model Fine-tuning Preview
Fine-tune a foundation model on your dataset
Start fine-tuning

Forecasting Preview
Input your dataset and create a forecasting model using AutoML
Start training

Classification
Input your dataset and create a classification model using AutoML
Start training

Regression
Input your dataset and create a regression model using AutoML
Start training

Experiments list

[Reset filters](#)

Name	Created by	Last modified	Location	Description
mlflow-classic-ml-e2e-mlflow-3 /		2025-06-05 09:37:20 PDT	/Users/	
mlflow-sklearn-quickstart		2025-06-03 14:22:15 PDT	/Users/	
mlflow-xgboost-quickstart		2025-06-03 14:17:27 PDT	/Users/	
mlflow-model-tracking-deep-learning		2025-05-29 16:24:50 PDT	/Users/	

Create workspace experiment

This section describes how to create a workspace experiment using the Databricks UI. You can create a workspace experiment directly [from the workspace](#) or [from the Experiments page](#).

You can also use the [MLflow API](#), or the [Databricks Terraform provider](#) with `databricks_mlflow_experiment`.

For instructions on logging runs to workspace experiments, see [Log runs and models to an experiment](#).

NOTE

For GDPR compliance, stale experiments are purged from workspaces. For workspaces where [customer-managed keys](#) are used, the stale experiments are only purged when the customer-managed key is accessible by Databricks. See [Configure customer-managed keys for encryption](#) to give access.

Create experiment from the workspace

1. Click  **Workspace** in the sidebar.
2. Navigate to the folder in which you want to create the experiment.
3. Right-click on the folder and select **Create > MLflow experiment**.
4. In the Create MLflow Experiment dialog, enter a name for the experiment and an optional artifact location. If you do not specify an artifact location, artifacts are stored in MLflow-managed artifact storage: `dbfs:/databricks/mlflow-tracking/<experiment-id>`.

For workspaces enabled for Unity Catalog, you can also store artifacts in a Unity Catalog volume. To store artifacts in your own cloud storage, create a Unity Catalog external volume.

To store artifacts in a Unity Catalog volume, specify a volume path of the form `dbfs:/Volumes/catalog_name/schema_name/volume_name/user/specified/path` as your MLflow experiment artifact location, either in the UI or as shown in the following code:

Python

```
import mlflow # Storing artifacts in a volume requires MLflow 2.15.0
or above

EXP_NAME = "/Users/first.last@databricks.com/my_experiment_name"
CATALOG = "my_catalog"
SCHEMA = "my_schema"
VOLUME = "my_volume"
ARTIFACT_PATH = f"dbfs:/Volumes/{CATALOG}/{SCHEMA}/{VOLUME}" # can be
a managed or external volume

mlflow.set_tracking_uri("databricks")
mlflow.set_registry_uri("databricks-uc")

if mlflow.get_experiment_by_name(EXP_NAME) is None:
    mlflow.create_experiment(name=EXP_NAME,
artifact_location=ARTIFACT_PATH)
mlflow.set_experiment(EXP_NAME)
```

If your workspace is not enabled for Unity Catalog, or you do not have access to MLflow 2.15.0 or above, specify a path in this format: `dbfs:/path/to/artifacts`.

Databricks recommends using a Unity Catalog volume for artifact storage. If neither a Unity Catalog volume nor DBFS is a suitable option, you can also store artifacts directly to S3 (not recommended). To store artifacts in S3, specify a URI of the form `s3://<bucket>/<path>`. MLflow obtains credentials to access S3 from your cluster's [instance profile](#). Artifacts stored in S3 do not appear in the MLflow UI; you must download them using an object storage client.

The upload and download file size limits are both 5GB.

NOTE

When you store an artifact in a location other than MLflow-managed DBFS (default) or Unity Catalog volumes, the artifact does not appear in the MLflow UI. Models stored in locations other than these cannot be registered in Model Registry.

5. Click **Create**. The experiment details page for the new experiment appears.

6. To log runs to this experiment, call `mlflow.set_experiment()` with the experiment path. To display the experiment path, click the information icon ⓘ to the right of the experiment name. See [Log runs and models to an experiment](#) for details and an example notebook.

Create experiment from the Experiments page

To create a foundation model fine-tuning, AutoML, or custom experiment, click **Experiments** or select **New > Experiment** in the left sidebar.

At the top of the page, select one of the following options to configure an experiment:

- **Foundation Model Fine-tuning.** The **Foundation Model Fine-tuning** dialog appears. For details, see [Create a training run using the Foundation Model Fine-tuning UI](#).
- **Forecasting.** The **Configure Forecasting experiment** dialog appears. For details, see [Configure the AutoML experiment](#).
- **Classification.** The **Configure Classification experiment** dialog appears. For details, see [Set up classification experiment with the UI](#).
- **Regression.** The **Configure Classification experiment** dialog appears. For details, see [Set up regression experiment with the UI](#).
- **Custom.** The **Create MLflow Experiment** dialog appears. For details, see Step 4 in [Create experiment from the workspace](#).

Create notebook experiment

When you use the `mlflow.start_run()` command in a notebook, the run logs metrics and parameters to the active experiment. If no experiment is active, Databricks creates a notebook experiment. A notebook experiment shares the same name and ID as its corresponding notebook. The notebook ID is the numerical identifier at the end of a [Notebook URL and ID](#).

ⓘ **NOTE**

 Ask Genie

Users running MLflow on compute with [dedicated group access](#) must verify the group has permission to write to the directory where the notebook lives, or use `mlflow.set_tracking_uri("<path>")` to specify a folder for MLflow to write to.

Alternatively, you can pass a Databricks workspace path to an existing notebook in `mlflow.set_experiment()` to create a notebook experiment for it.

For instructions on logging runs to notebook experiments, see [Log runs and models to an experiment](#).

NOTE

If you delete a notebook experiment using the API (for example, `MlflowClient.tracking.delete_experiment()` in Python), the notebook itself is moved into the Trash folder.

View experiments

Each experiment that you have access to appears on the experiments page. From this page, you can view any experiment. Click on an experiment name to display the experiment details page.

Additional ways to access the experiment details page:

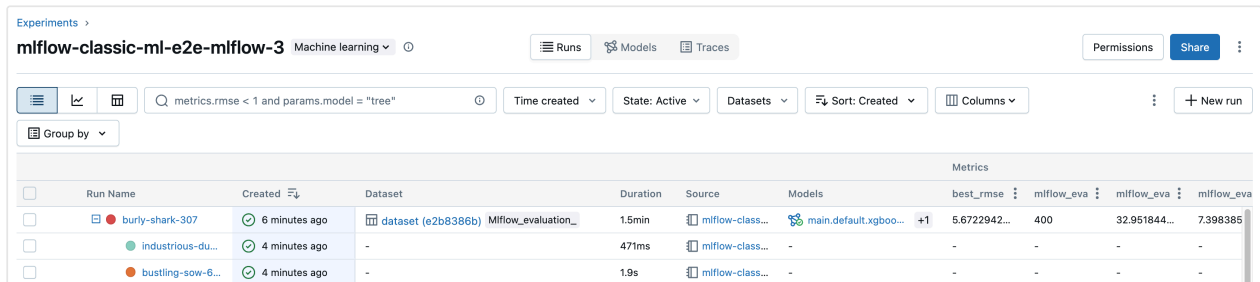
- You can access the experiment details page for a workspace experiment from the workspace menu.
- You can access the experiment details page for a notebook experiment from the notebook.

To search for experiments, type text in the **Filter experiments** field and press **Enter** or click the magnifying glass icon. The experiment list changes to show only those experiments that contain the search text in the **Name** or **Location** columns.

For advanced usage, you can enter a search query for `tags.`mlflow.note.content`` to search based on the **Description** column. For further details on syntax, see [Search Experiments](#). Please note that unlike searching for **Name** or **Location**, searching

through tags requires you to manually construct the search query with an identifier and comparator. It will not directly return all results that contain the search text.

Click the name of any experiment in the table to display its experiment details page:



The screenshot shows the MLflow Experiments page for an experiment named 'mflow-classic-ml-e2e-mflow-3'. The interface includes a search bar with the query 'metrics.rmse < 1 and params.model = "tree"', filters for 'Time created', 'State: Active', and 'Datasets', and a '+ New run' button. Below the filters is a table of runs.

Run Name	Created	Dataset	Duration	Source	Models	best_rmse	mlflow_eva	mlflow_eva	mlflow_eva
burly-shark-307	6 minutes ago	dataset (e2b8386b) Mlflow_evaluation...	1.5min	mflow-class...	main.default.xgboo...	+1 5.6722942...	400	32.951844...	7.398385
industrious-du...	4 minutes ago	-	471ms	mflow-class...	-	-	-	-	-
bustling-sow-6...	4 minutes ago	-	1.9s	mflow-class...	-	-	-	-	-

The experiment details page lists all runs associated with the experiment. From the table, you can open the run page for any run associated with the experiment by clicking its **Run Name**. The **Source** column gives you access to the notebook version that created the run. You can also search and [filter](#) runs by metrics or parameter settings.

View workspace experiment

1. Click  **Workspace** in the sidebar.
2. Go to the folder containing the experiment.
3. Click the experiment name.

View notebook experiment

In the notebook's right sidebar, click the **Experiment** icon .

The Experiment Runs sidebar appears and shows a summary of each run associated with the notebook experiment, including run parameters and metrics. At the top of the sidebar is the name of the experiment that the notebook most recently logged runs to (either a notebook experiment or a workspace experiment).

MLflow quickstart (Python) Python ☆

File Edit View Run Help ▶ Run all 15.4 LTS ML Schedule Share

MLflow quickstart (Python)

With MLflow's autologging capabilities, a single line of code automatically logs the resulting model, the parameters used to create the model, and a model score. MLflow autologging is available for several widely used machine learning packages. This notebook creates a Random Forest model on a simple dataset and uses the MLflow `autolog()` function to log information generated by the run.

For details about what information is logged with `autolog()`, refer to the [MLflow documentation](#).

Setup

- If you are using a cluster running Databricks Runtime, you must install the mlflow library from PyPI. See Cmd 3.
- If you are using a cluster running Databricks Runtime ML, the mlflow library is already installed.

Install the mlflow library. This is required for Databricks Runtime clusters only. If you are using a cluster running Databricks Runtime ML, skip to Cmd 4.

MLflow experiments

MLflow quickstart (Python)

Experiment Runs Date ↓ ↺ ↻

casual-lamb-787

2024-12-09 16:31:33 PST

bootstrap: True, ccp_alpha: 0.0, ...

training_mean_absolute_error: 40.09, ...

Models sklearn

capable-bear-725

2024-12-09 16:30:54 PST

bootstrap: True, ccp_alpha: 0.0, ...

training_mean_absolute_error: 41.1, ...

Models sklearn

Showing 2 runs, for more information go to [Experiment UI](#)

From the sidebar, you can navigate to the experiment details page or directly to a run.

- To view the experiment, click  at the far right, next to **Experiment Runs**.
- To display a [run](#), click the name of the run.

Manage experiments

You can rename, delete, or manage permissions for an experiment you own from the experiments page, the [experiment details page](#), or the workspace menu.

NOTE

You cannot directly rename, delete, or manage permissions on an MLflow experiment that was created by a notebook in a Databricks Git folder. You must perform these actions at the Git folder level.

Rename experiment

You can rename an experiment that you own from the [Experiments page](#) or from the experiment details page for that experiment.

- On the **Experiments** page, click the kebab menu  in the rightmost column and then click **Rename**.

Experiments Compare (0)

Create model training

GenAI apps & agents
 Build a Generative AI application or agent
 Create experiment

Foundation Model Fine-tuning Preview
 Fine-tune a foundation model on your dataset
 Start fine-tuning

Forecasting Preview
 Input your dataset and create a forecasting model using AutoML
 Start training

Classification
 Input your dataset and create a classification model using AutoML
 Start training

Regression
 Input your dataset and create a regression model using AutoML
 Start training

Experiments list

mlflow-

Name	Created by	Last modified	Location	Description	
mlflow-classic-ml-e2e-mlflow-3 /		2025-06-05 09:37:20 PDT	/Users/		
mlflow-sklearn-quickstart		2025-06-03 14:22:15 PDT	/Users/		Rename
mlflow-xgboost-quickstart		2025-06-03 14:17:27 PDT	/Users/		Permissions
mlflow-model-tracking-deep-learning		2025-05-29 16:24:50 PDT	/Users/		Delete

- On the experiment details page, click the kebab menu next to **Permissions** and then click **Rename**.

Experiments > **mlflow-classic-ml-e2e-mlflow-3** Machine learning

You can rename a workspace experiment from the workspace. Right-click the experiment name and then click **Rename**.

Get experiment ID and path to experiment

On the experiment details page, you can get the path to a notebook experiment by clicking the information icon to the right of the experiment name. A pop-up note appears that shows the path to the experiment, the experiment ID, and the artifact location. You can use the experiment ID in the MLflow command `set_experiment` to set the active MLflow experiment.

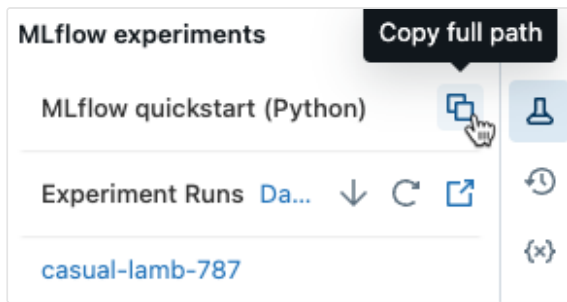
Experiments > **mlflow-classic-ml-e2e-mlflow-3** Machine learning

Path: /Users/ /mlflow-classic-ml-e2e-mlflow-3

Experiment ID: 601321712471756

Artifact Location: dbfs:/databricks/mlflow-tracking/601321712471756

From a notebook, you can copy the full path of the experiment by clicking in the notebook's experiment sidebar.



Delete notebook experiment

Notebook experiments are part of the notebook and cannot be deleted separately. When you [delete a notebook](#), the associated notebook experiment is deleted. When you delete a notebook experiment using the UI, the notebook is also deleted.

To delete notebook experiments using the API, use the [Workspace API](#) to ensure both the notebook and experiment are deleted from the workspace.

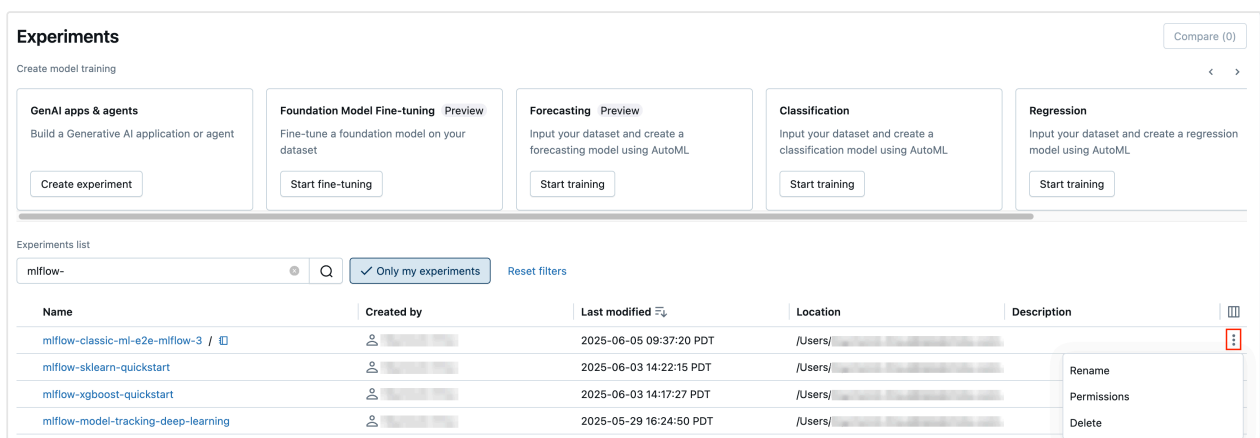
Delete a workspace or notebook experiment

You can delete an experiment that you own from the [experiments page](#) or from the experiment details page.

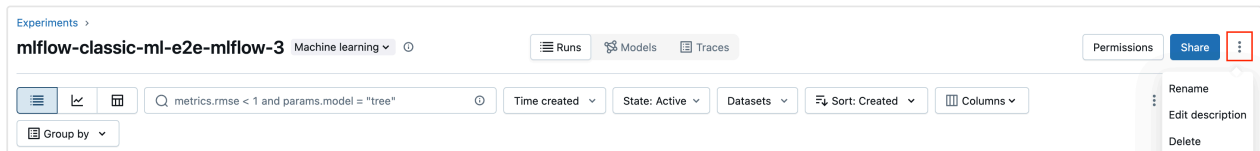
ⓘ IMPORTANT

When you delete a notebook experiment, the notebook is also deleted.

- On the **Experiments** page, click the kebab menu  in the rightmost column and then click **Delete**.



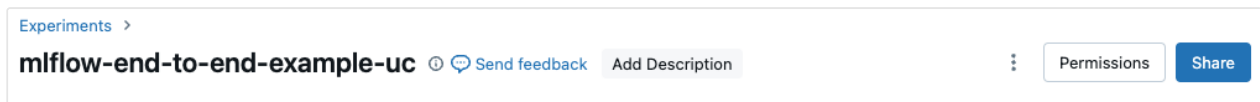
- On the experiment details page, click the kebab menu  next to **Permissions** and then click **Delete**.



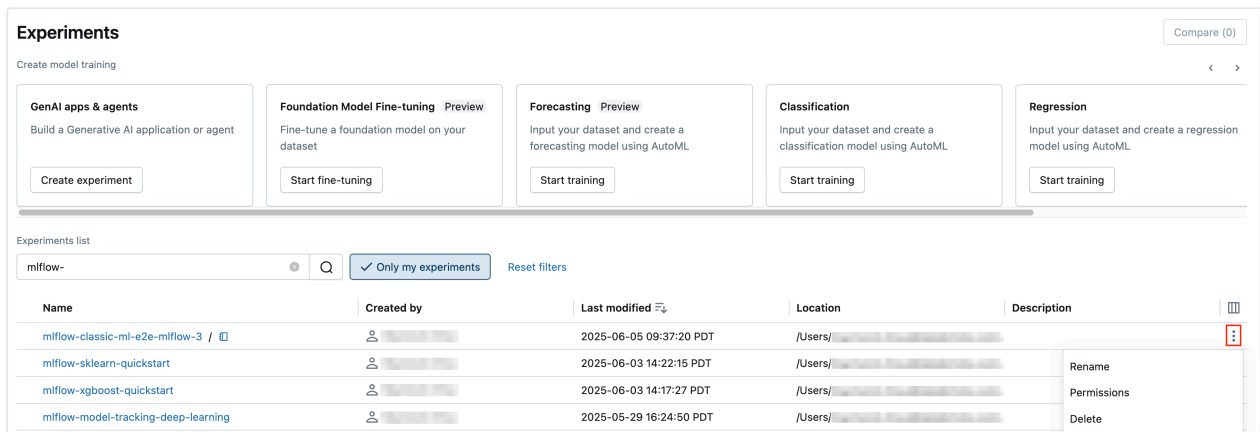
You can delete a workspace experiment from the workspace. Right-click the experiment name and then click **Move to Trash**.

Change permissions for an experiment

To change permissions for an experiment from the [experiment details page](#), click **Permissions**.



You can change permissions for an experiment that you own from the [Experiments page](#). Click the kebab menu  in the rightmost column and then click **Permissions**.



For information on experiment permission levels, see [MLflow experiment ACLs](#).

Copy experiments between workspaces

To migrate MLflow experiments between workspaces, you can use the community-driven open source project [MLflow Export-Import](#).

With these tools, you can:

- Share and collaborate with other data scientists in the same or another tracking server. For example, you can clone an experiment from another user into your workspace.
- Copy MLflow experiments and runs from your local tracking server to your Databricks workspace.
- Back up mission critical experiments and models to another Databricks workspace.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Feb 25, 2026**

Choose where your MLflow data is stored

MLflow tracking servers store and manage your experiment data, runs, and models. Configure your tracking servers to control where your MLflow data is stored and how to access experiments across different environments.

Databricks-hosted tracking server

By default, Databricks provides a managed MLflow tracking server that:

- Requires no additional setup or configuration
- Stores experiment data in your workspace
- Integrates seamlessly with Databricks notebooks and clusters

Set the active experiment

By default all MLflow runs are logged to workspace's tracking server using the active experiment. If no experiment is explicitly set, runs are logged to the [notebook experiment](#).

Control where runs are logged in Databricks by setting the active experiment:

[MLflow.set_experiment\(\)](#)

[MLflow.start_run\(\)](#)

[Environment variables](#)

Set an experiment for all subsequent runs in the execution.

Python

```
import mlflow

mlflow.set_experiment("/Shared/my-experiment")
```



Set up tracking to a remote MLflow tracking server

You may need to set up a connection to a remote MLflow tracking server. This could be because you are developing locally and want to track against the Databricks hosted server, or you want to track to a different MLflow tracking server. For example, one that's in a different workspace.

Common scenarios for remote tracking:

Scenario	Use Case
Cross-workspace tracking	Centralized experiment tracking across multiple workspaces
Local development	Develop locally but track experiments in Databricks
Remote self-hosted	Custom MLflow infrastructure with specific compliance requirements

Set up the tracking URI and experiment

To log experiments to a remote tracking server, configure both the tracking URI and experiment path:

Python

```
import mlflow

# Set the tracking URI to the remote server
```

 Ask Genie

```
mlflow.set_tracking_uri("databricks://remote-workspace-url")

# Set the experiment path in the remote server
mlflow.set_experiment("/Shared/centralized-experiments/my-project")

# All subsequent runs will be logged to the remote server
with mlflow.start_run():
    mlflow.log_param("model_type", "random_forest")
    mlflow.log_metric("accuracy", 0.95)
```

Authentication methods

Remote tracking server connections require proper authentication. Choose between Personal Access Tokens (PAT) or OAuth using service principals.

PAT **OAuth (service principal)**

Use PATs for simple token-based authentication.

Pros: Simple setup, good for development

Cons: User-specific, requires manual token management

Python

```
import os

# Set authentication token
os.environ["DATABRICKS_TOKEN"] = "your-personal-access-token"

# Configure remote tracking
mlflow.set_tracking_uri("databricks://remote-workspace-url")
mlflow.set_experiment("/Shared/remote-experiment")
```


Last updated on **Feb 25, 2026**

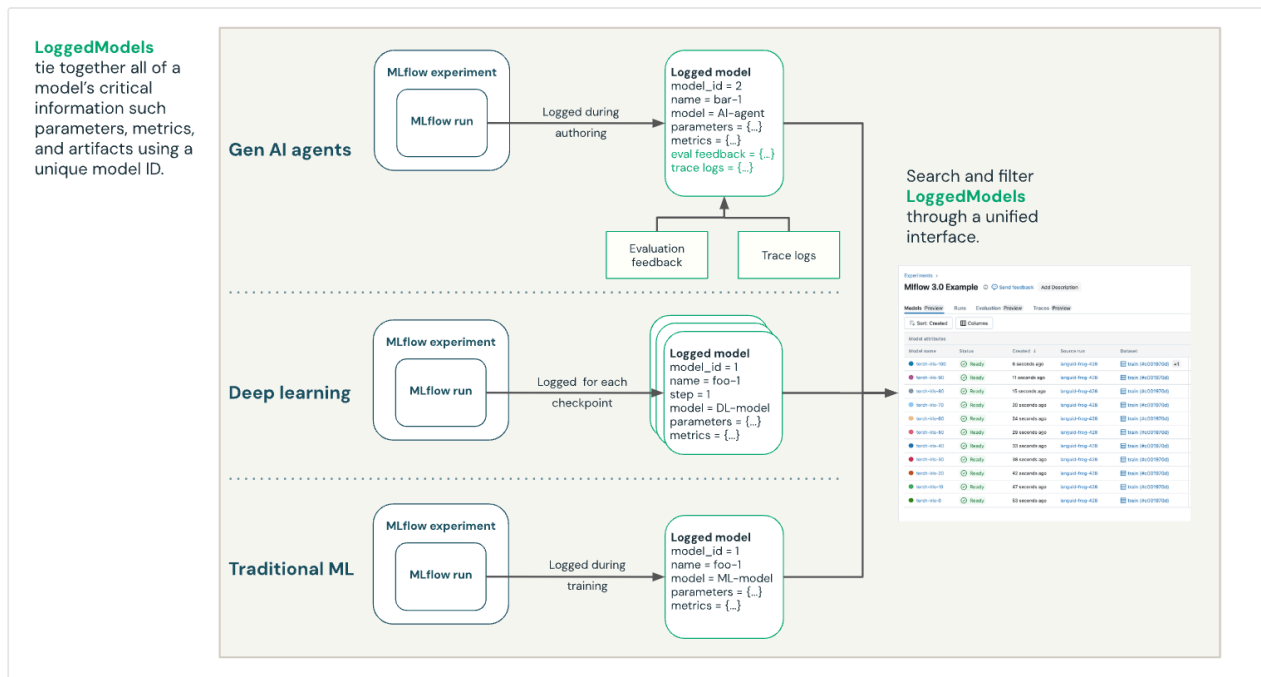
Track and compare models using MLflow Logged Models

MLflow Logged Models help you track a model's progress throughout its lifecycle. When you train a model, use `mlflow.<model-flavor>.log_model()` to create a `LoggedModel` that ties together all of its critical information using a unique ID. To leverage the power of `LoggedModels`, get started with [MLflow 3](#).

For GenAI applications, `LoggedModels` can be created to capture git commits or sets of parameters as dedicated objects that can then be linked to traces and metrics. In deep learning and classical ML, `LoggedModels` are produced from MLflow runs, which are existing concepts in MLflow and can be thought of as jobs that execute model code. Training runs produce models as outputs, and evaluation runs use existing models as input to produce metrics and other information you can use to assess the performance of a model.

The `LoggedModel` object persists throughout the model's lifecycle, across different environments, and contains links to artifacts such as metadata, metrics, parameters, and the code used to generate the model. Logged Model tracking lets you compare models against each other, find the most performant model, and track down information during debugging.

Logged Models can also be registered to the Unity Catalog model registry, making information about the model from all MLflow experiments and workspaces available in a single location. For more details, see [Model Registry improvements with MLflow 3](#).



Improved tracking for gen AI and deep learning models

Generative AI and deep learning workflows especially benefit from the granular tracking that Logged Models provides.

Gen AI – unified evaluation and trace data:

- Gen AI models generate additional metrics during evaluation and deployment, such as reviewer feedback data and traces.
- The `LoggedModel` entity allows you to query all the information generated by a model using a single interface.

Deep learning – efficient checkpoint management:

- Deep learning training creates multiple checkpoints, which are snapshots of the model's state at a particular point during training.
- MLflow creates a separate `LoggedModel` for each checkpoint, containing the model's metrics and performance data. This allows you to compare and evaluate checkpoints to identify the best-performing models efficiently.

Create a Logged Model

To create a Logged Model, use the same `log_model()` API as existing MLflow workloads. The following code snippets show how to create a Logged Model for gen AI, deep learning, and traditional ML workflows.

For complete, runnable notebook examples, see [Example notebooks](#).

Gen AI **Deep learning** **Traditional ML**

The following code snippet shows how to log a LangChain agent. Use the `log_model()` method for your flavor of agent.

Python

```
# Log the chain with MLflow, specifying its parameters
# As a new feature, the LoggedModel entity is linked to its name and
params
model_info = mlflow.langchain.log_model(
    lc_model=chain,
    name="basic_chain",
    params={
        "temperature": 0.1,
        "max_tokens": 2000,
        "prompt_template": str(prompt)
    },
    model_type="agent",
    input_example={"messages": "What is MLflow?"},
)

# Inspect the LoggedModel and its properties
logged_model = mlflow.get_logged_model(model_info.model_id)
print(logged_model.model_id, logged_model.params)
```

Start an evaluation job and link the metrics to a Logged Model by providing the unique `model_id` for the `LoggedModel`:

Python

```
# Start a run to represent the evaluation job
with mlflow.start_run() as evaluation_run:
```

 Ask Genie

```

eval_dataset: mlflow.entities.Dataset = mlflow.data.from_pandas(
    df=eval_df,
    name="eval_dataset",
)
# Run the agent evaluation
result = mlflow.evaluate(
    model=f"models:{logged_model.model_id}",
    data=eval_dataset,
    model_type="databricks-agent"
)
# Log evaluation metrics and associate with agent
mlflow.log_metrics(
    metrics=result.metrics,
    dataset=eval_dataset,
    # Specify the ID of the agent logged above
    model_id=logged_model.model_id
)

```

Example notebooks

For example notebooks that illustrate the use of `LoggedModels`, see the following pages:

- [MLflow 3 deep learning workflow](#)
- [MLflow 3 traditional ML workflow](#)

View models and track progress

You can view your Logged Models in the workspace UI:

1. Go to the **Experiments** tab in your workspace.
2. Select an experiment. Then, select the **Models** tab.

This page contains all the Logged Models associated with the experiment, along with their metrics, parameters, and artifacts.

New

Workspace

Recents

Catalog

Discover

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

Playground

Agents Beta

Experiments

Features

Models

Experiments >

mlflow-model-tracking-deep-le... Machine learning

Runs

Models

Traces

Permissions

Share

metrics.rmse >= 0.8

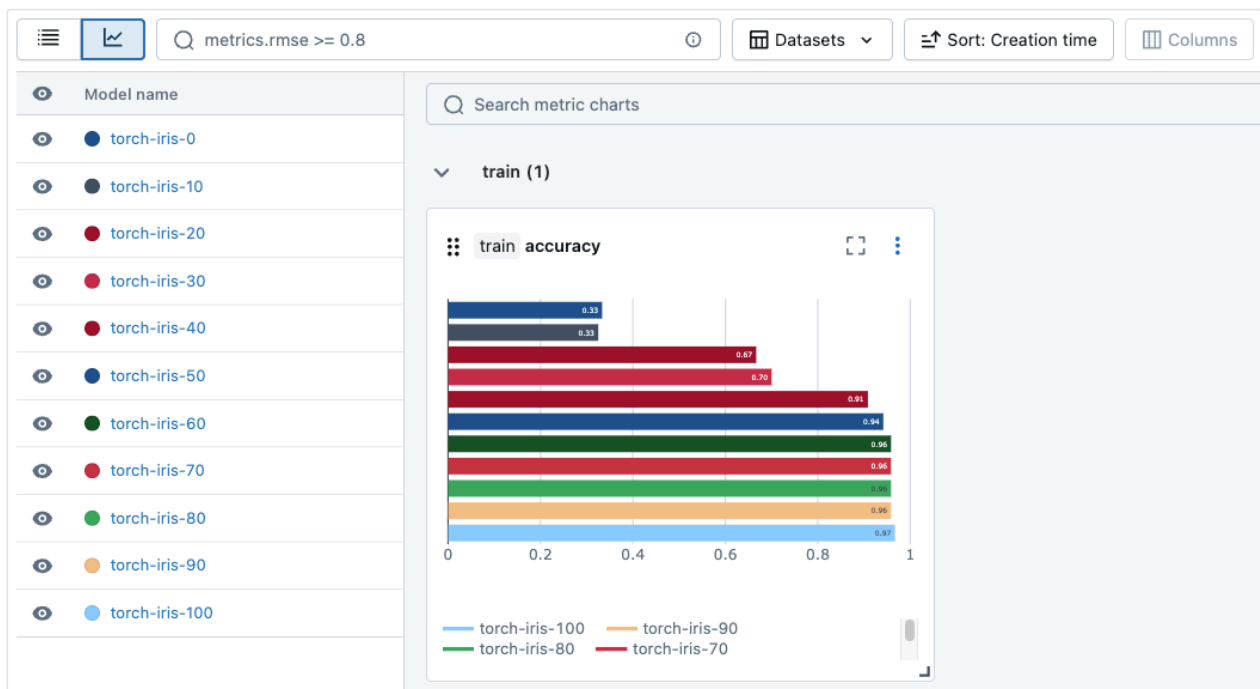
Datasets

Sort: Creation time

Columns

Model attributes			Model attributes			Dataset: train (#1f1c13b5)		Parameters		
Model name	Status	Created	Logged from	Source run	Dataset	accuracy	activation	criterion	n_layers	
torch-iris-100	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9666666666666667	ReLU	CrossEntropyLoss	3	
torch-iris-90	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3	
torch-iris-80	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3	
torch-iris-70	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3	
torch-iris-60	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9583333333333334	ReLU	CrossEntropyLoss	3	
torch-iris-50	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9416666666666667	ReLU	CrossEntropyLoss	3	
torch-iris-40	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.9083333333333333	ReLU	CrossEntropyLoss	3	
torch-iris-30	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.7	ReLU	CrossEntropyLoss	3	
torch-iris-20	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.6666666666666666	ReLU	CrossEntropyLoss	3	
torch-iris-10	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.325	ReLU	CrossEntropyLoss	3	
torch-iris-0	Ready	5 days ago	mlflow3-di-example	traveling-gnu-840	train (#1f1c13b5)	0.3333333333333333	ReLU	CrossEntropyLoss	3	

You can generate charts to track metrics across runs.



Search and filter Logged Models

From the **Models** tab, you can search and filter Logged Models based on their attributes, parameters, tags, and metrics.

Experiments > **mlflow3-dl-example** [Send feedback](#) [Add Description](#) Permissions [Share](#)

Runs **Models** Preview Evaluation Monitoring Beta Traces

Model attributes				train (#1f1c13b5)		Dataset: train (#1f1c13b5)	Parameters
Model name	Status	Created ↓	Logged from	Source run	Dataset	accuracy	activation
torch-iris-100	Ready	7 minutes ago	mlflow3-dl-example	learned-finch-807	train (#1f1c13b5)	0.975	ReLU
torch-iris-90	Ready	7 minutes ago	mlflow3-dl-example	learned-finch-807	train (#1f1c13b5)	0.975	ReLU
torch-iris-80	Ready	7 minutes ago	mlflow3-dl-example	learned-finch-807	train (#1f1c13b5)	0.975	ReLU
torch-iris-70	Ready	7 minutes ago	mlflow3-dl-example	learned-finch-807	train (#1f1c13b5)	0.975	ReLU
torch-iris-60	Ready	8 minutes ago	mlflow3-dl-example	learned-finch-807	train (#1f1c13b5)	0.9833333333333333	ReLU
torch-iris-50	Ready	8 minutes ago	mlflow3-dl-example	learned-finch-807	train (#1f1c13b5)	0.975	ReLU
torch-iris-40	Ready	8 minutes ago	mlflow3-dl-example	learned-finch-807	train (#1f1c13b5)	0.9666666666666667	ReLU

You can filter metrics based on dataset-specific performance, and only models with matching metric values on the given datasets are returned. If dataset filters are provided without any metric filters, models with *any* metrics on those datasets are returned.

You can filter based on the following attributes:

- `model_id`
- `model_name`
- `status`
- `artifact_uri`
- `creation_time` (numeric)
- `last_updated_time` (numeric)

Use the following operators to search and filter string-like attributes, parameters, and tags:

- `=`, `!=`, `IN`, `NOT IN`

Use the following comparison operators to search and filter numeric attributes and metrics:

- `=`, `!=`, `>`, `<`, `>=`, `<=`

Search Logged Models programmatically

You can search for Logged Models using the MLflow API:

```

## Get a Logged Model using a model_id
mlflow.get_logged_model(model_id = <my-model-id>)

## Get all Logged Models that you have access to
mlflow.search_logged_models()

## Get all Logged Models with a specific name
mlflow.search_logged_models(
    filter_string = "model_name = <my-model-name>"
)

## Get all Logged Models created within a certain time range
mlflow.search_logged_models(
    filter_string = "creation_time >= <creation_time_start> AND
creation_time <= <creation_time_end>"
)

## Get all Logged Models with a specific param value
mlflow.search_logged_models(
    filter_string = "params.<param_name> = <param_value_1>"
)

## Get all Logged Models with specific tag values
mlflow.search_logged_models(
    filter_string = "tags.<tag_name> IN (<tag_value_1>, <tag_value_2>)"
)

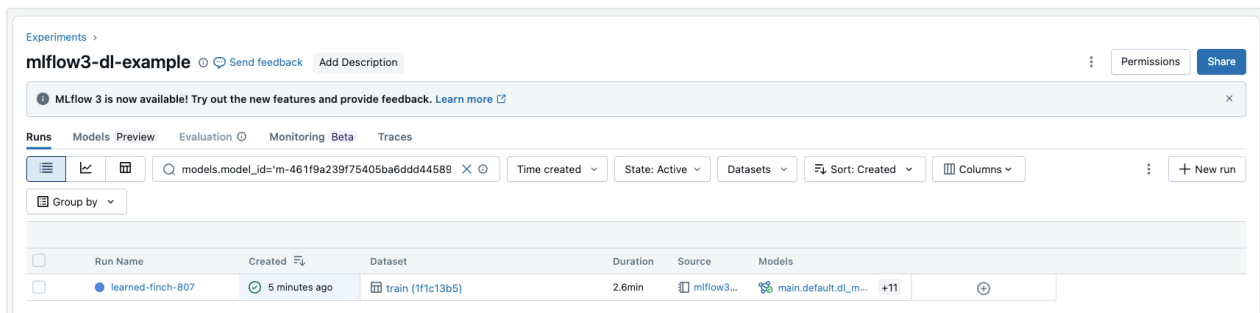
## Get all Logged Models greater than a specific metric value on a
dataset, then order by that metric value
mlflow.search_logged_models(
    filter_string = "metrics.<metric_name> >= <metric_value>",
    datasets = [
        {"dataset_name": <dataset_name>, "dataset_digest": <dataset_digest>}
    ],
    order_by = [
        {"field_name": metrics.<metric_name>, "dataset_name":
<dataset_name>, "dataset_digest": <dataset_digest>}
    ]
)

```

For more information and additional search parameters see the [MLflow 3 API documentation](#).

Search runs by model inputs and outputs Ask Genie

You can search runs by model ID to return all runs that have the Logged Model as an input or output. For more information on the filter string syntax, see [filtering for runs](#).



You can search for runs using the MLflow API:

Python

```
## Get all Runs with a particular model as an input or output by model id
mlflow.search_runs(filter_string = "models.model_id = <my-model-id>")
```

Next steps

To learn more about other new features of MLflow 3, see the following articles:

- [Model Registry improvements with MLflow 3](#).
- [MLflow 3 deployment jobs](#).

Is this page

as this page helpful?

☒ Yes

☐ No

☐ Send feedback

View training results with MLflow runs

Last updated on **Feb 25, 2026**

View training results with MLflow runs

This article describes how to use MLflow runs to view and analyze the results of a model training experiment, and how to manage and organize runs. For more information about MLflow experiments, see [Organize training runs with MLflow experiments](#).

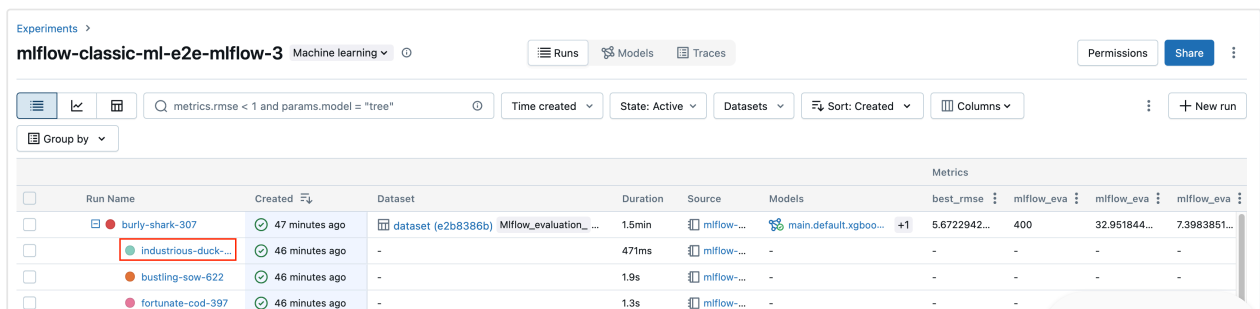
An MLflow *run* corresponds to a single execution of model code. Each run records information such as the notebook that launched the run, any models created by the run, model parameters and metrics saved as key-value pairs, tags for run metadata, and any artifacts, or output files, created by the run.

All MLflow runs are logged to the [active experiment](#). If you have not explicitly set an experiment as the active experiment, runs are logged to the notebook experiment.

View run details

You can access a run either from its experiment details page or directly from the notebook that created the run.

From the [experiment details page](#), click the run name in the runs table.

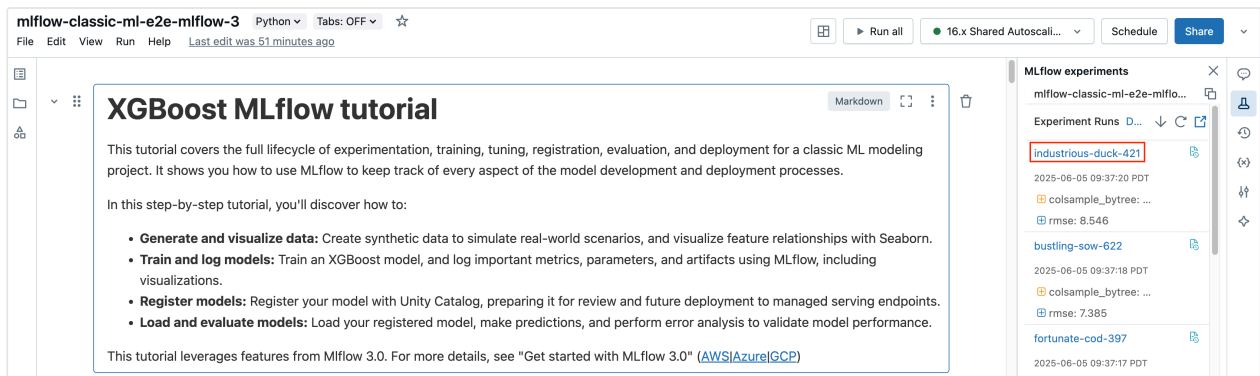


The screenshot shows the MLflow Experiments page for the experiment 'mlflow-classic-ml-e2e-mlflow-3'. The interface includes a search bar with the query 'metrics.rmse < 1 and params.model = "tree"', filters for 'Time created', 'State: Active', and 'Datasets', and a table of runs. The run 'industrious-duck-...' is highlighted with a red box.

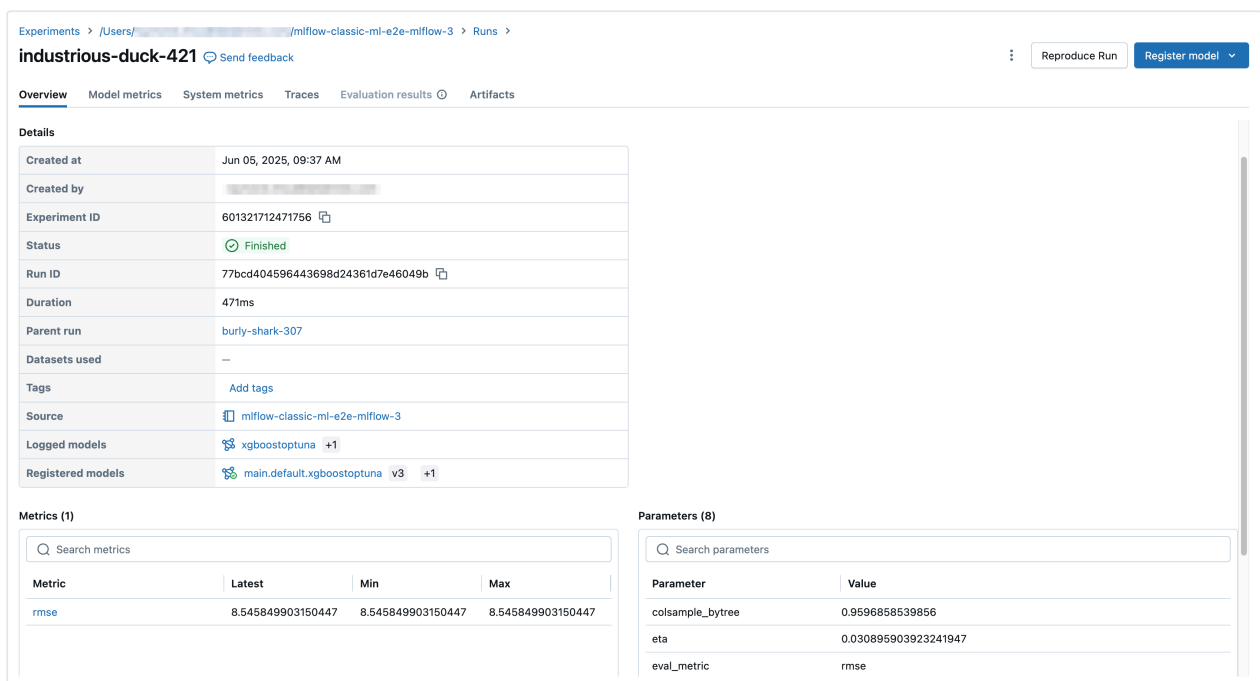
Run Name	Created	Dataset	Duration	Source	Models	best_rmse	mlflow_eva	mlflow_eva	mlflow_eva
burly-shark-307	47 minutes ago	dataset (e2b8386b)	1.5min	mlflow-...	main.default.xgboo...	5.6722942...	400	32.951844...	7.3983851...
industrious-duck-...	46 minutes ago	-	471ms	mlflow-...	-	-	-	-	-
bustling-sow-622	46 minutes ago	-	1.9s	mlflow-...	-	-	-	-	-
fortunate-cod-397	46 minutes ago	-	1.3s	mlflow-...	-	-	-	-	-

Ask Genie

From the notebook, click the run name in the Experiment Runs sidebar.



The [run screen](#) shows the run ID, the parameters used for the run, the metrics resulting from the run, and details about the run including a link to the source notebook. Artifacts saved from the run are available in the **Artifacts** tab.



Code snippets for prediction

If you log a model from a run, the model appears in the **Artifacts** tab, along with code snippets illustrating how to load and use the model to make predictions on Spark and Pandas DataFrames. In [MLflow 3](#), models are now their distinct first-class object rather than being logged as run artifact. For more information, see [Get started with MLflow 3 for models](#).

Experiments > /Users/ > /mlflow-end-to-end-example-uc >

melodic-koi-187
Send feedback

Reproduce Run
Register model

Overview
Model metrics
System metrics
Traces
Evaluation results
Preview
Artifacts

model

- metadata
 - MLmodel
 - conda.yaml
 - model.xgb
 - python_env.yaml
 - requirements.txt
- feature_importance_weight.json
- feature_importance_weight.png

model

Path: dbfs:/databricks/mlflow-tracking/1367206172064334/8df4fe97be0542b2840aeb7e528f3513/artifacts/model

Register model

MLflow Model

The code snippets below demonstrate how to make predictions using the logged model. This model is also registered to the [model registry](#).

Model schema

Input and output schema for your model. [Learn more](#)

Name	Type
Inputs (12)	
fixed_acidity (required)	double
volatile_acidity (required)	double
citric_acid (required)	double
Outputs (1)	
- (required)	Tensor (dtype: float32, shape: [-1])

Validate the model before deployment

Run the following code to validate model inference works on the example payload, prior to deploying it to a serving endpoint

```

from mlflow.models import validate_serving_input

model_uri = 'runs:/8df4fe97be0542b2840aeb7e528f3513/model'

# The logged model does not contain an input_example.
# Manually generate a serving payload to verify your model prior to deployment.
from mlflow.models import convert_input_example_to_serving_input

# Define INPUT_EXAMPLE via assignment with your own input example to the model
# A valid input example is a data instance suitable for pyfunc prediction
serving_payload = convert_input_example_to_serving_input(INPUT_EXAMPLE)

# Validate the serving payload works on the model
validate_serving_input(model_uri, serving_payload)

```

Make Predictions

Predict on a Pandas DataFrame:

```

import mlflow
logged_model = 'runs:/8df4fe97be0542b2840aeb7e528f3513/model'

# Load model as a PyFuncModel.
loaded_model = mlflow.pyfunc.load_model(logged_model)

# Predict on a Pandas DataFrame.
import pandas as pd
loaded_model.predict(pd.DataFrame(data))

```

Predict on a Spark DataFrame:

```

import mlflow
from pyspark.sql.functions import struct, col
logged_model = 'runs:/8df4fe97be0542b2840aeb7e528f3513/model'

```

View the notebook used for a run

To view the [version of the notebook](#) that created a run:

- On the experiment details page, click the link in the **Source** column.
- On the run page, click the link next to **Source**.
- From the notebook, in the Experiment Runs sidebar, click the **Notebook** icon  in the box for that Experiment Run.

The version of the notebook associated with the run appears in the main window with a highlight bar showing the date and time of the run.

 Ask Genie

Add a tag to a run

Tags are key-value pairs that you can create and use later to [search for runs](#).

1. In the **Details** table on the [run page](#), click **Add tags** next to **Tags**.

Details	
Created at	Jun 05, 2025, 09:37 AM
Created by	
Experiment ID	601321712471756
Status	Finished
Run ID	77bcd404596443698d24361d7e46049b
Duration	471ms
Parent run	burly-shark-307
Datasets used	—
Tags	Add tags
Source	mlflow-classic-ml-e2e-mlflow-3
Logged models	xgboostoptuna +1
Registered models	main.default.xgboostoptuna v3 +1

2. The Add/Edit tags dialog opens. In the **Key** field, enter a name for the key, and click **Add tag**.

Add/Edit tags ×

Key

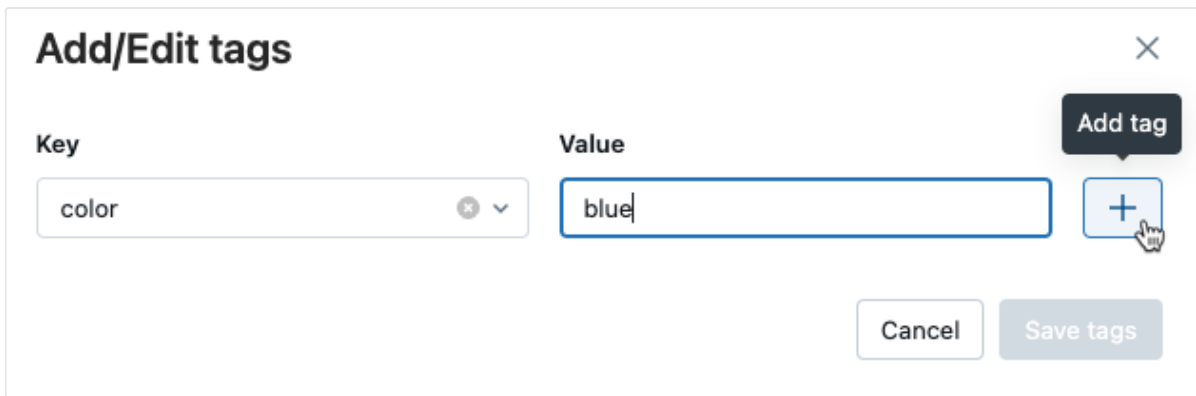
Value

Add tag "color"

Cancel

Save tags

3. In the **Value** field, enter the value for the tag.
4. Click the plus sign to save the key-value pair you just entered.



Add/Edit tags [X]

Key [X] [v]

Value

[Add tag] [+]

[Cancel] [Save tags]

5. To add additional tags, repeat steps 2 through 4.

6. When you are done, click **Save tags**.

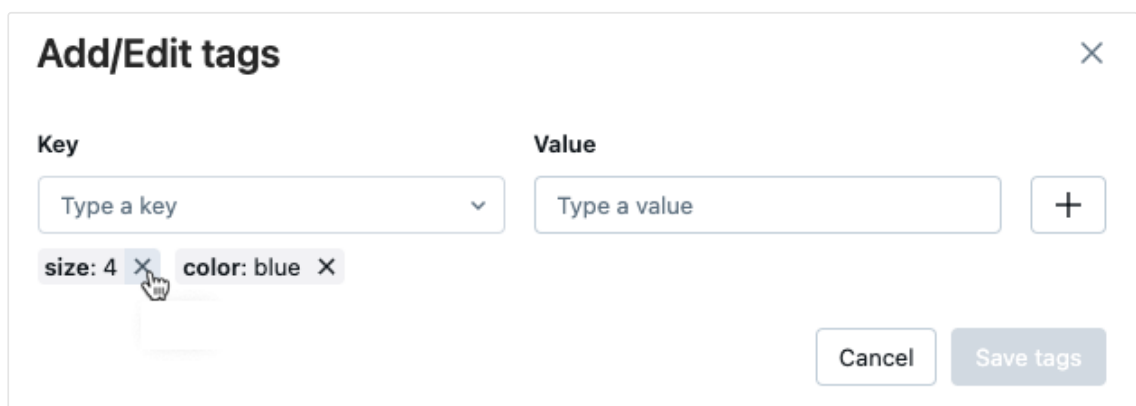
Edit or delete a tag for a run

1. In the **Details** table on the [run page](#), click  next to the existing tags.

Datasets used	—
Tags	size: 4 color: blue 
Source	 mlflow-quick-start-training

2. The Add/Edit tags dialog opens.

i. To delete a tag, click the X on that tag.



Add/Edit tags [X]

Key [v]

Value

[+]

size: 4 [X] color: blue [X]

[Cancel] [Save tags]

ii. To edit a tag, select the key from the drop-down menu, and edit the value in the **Value** field. Click the plus sign to save your change.

Add/Edit tags

Key

Type a key

size
color

Value

Type a value

+

Cancel

Save tags

3. When you are done, click **Save tags**.

Reproduce the software environment of a run

You can reproduce the exact software environment for the run by clicking **Reproduce Run** at the upper-right of the run page. The following dialog appears:

Reproduce Run

You can reproduce a run to recreate and iterate on the original notebook and cluster.

☒

**Clone Notebook**
Clone [MLflowPythonQuickstart](#) to
/Users/

Edit Folder

☒

**Recreate cluster** and attach notebook ?

View Spec

☒

**Install cluster libraries** ?

View Libraries

Cancel

Confirm

With the default settings, when you click **Confirm**:

- The notebook is cloned to the location shown in the dialog.

- If the original cluster still exists, the cloned notebook is attached to the original cluster and the cluster is started.
- If the original cluster no longer exists, a new cluster with the same configuration, including any installed libraries, is created and started. The notebook is attached to the new cluster.

You can select a different location for the cloned notebook and inspect the cluster configuration and installed libraries:

- To select a different folder to save the cloned notebook, click **Edit Folder**.
- To see the cluster spec, click **View Spec**. To clone only the notebook and not the cluster, uncheck this option.
- If the original cluster no longer exists, you can see the libraries installed on the original cluster by clicking **View Libraries**. If the original cluster still exists, this section is grayed out.

Rename run

To rename a run, click the kebab menu  at the upper right corner of the run page (next to the **Permissions** button) and select **Rename**.



Select columns to display

To control the columns displayed in the runs table on the experiment details page, click **Columns** and select from the drop-down menu.

Filter runs

You can search for runs in the table on the experiment details page based on parameter or metric values. You can also search for runs by tag.

- To search for runs that match an expression containing parameter and metric values, enter a query in the search field and press Enter. Some query syntax examples are:

```
metrics.r2 > 0.3
```

```
params.elasticNetParam = 0.5
```

```
params.elasticNetParam = 0.5 AND metrics.avg_areaUnderROC > 0.3
```

```
MIN(metrics.rmse) <= 1
```

```
MAX(metrics.memUsage) > 0.9
```

```
LATEST(metrics.memUsage) = 0 AND MIN(metrics.rmse) <= 1
```

By default, metric values are filtered based on the last logged value. Using `MIN` or `MAX` lets you search for runs based on the minimum or maximum metric values, respectively. Only runs logged after August 2024 have minimum and maximum metric values.

- To search for runs by tag, enter tags in the format: `tags.<key>=<value>`. String values must be enclosed in quotes as shown.

```
tags.estimator_name="RandomForestRegressor"
```

```
tags.color="blue" AND tags.size=5
```

Both keys and values can contain spaces. If the key includes spaces, you must enclose it in backticks as shown.

```
tags.`my custom tag` = "my value"
```

You can also filter runs based on their state (Active or Deleted), when the run was created, and what datasets were used. To do this, make your selections from the **Time created**, **State**, or **Datasets** drop-down menus respectively.

Experiments > **mlflow-classic-ml-e2e-mlflow-3** Machine learning

Runs Models Traces Permissions Share

Group by

metrics.rmse < 1 and params.model = "tree" Time created State: Active Datasets Sort: Created Columns + New run

Run Name	Created	Dataset	Duration	Source	Models	Metrics
burly-shark-307	6 minutes ago	dataset (e2b8386b) Mlflow_evaluation...	1.5min	mlflow-class...	main.default.xgboo... +1	best_rmse 5.6722942... mflow_eva 400 mflow_eva 32.951844... mflow_eva 7.398385
industrious-du...	4 minutes ago	-	471ms	mlflow-class...	-	- - - -
bustling-sow-6...	4 minutes ago	-	1.9s	mlflow-class...	-	- - - -

Download runs

You can download runs from the Experiment details page as follows:

1. Click  to open the kebab menu.

Experiments > **mlflow-classic-ml-e2e-mlflow-3** Machine learning

Runs Models Traces Permissions Share

Group by

metrics.rmse < 1 and params.model = "tree" Time created State: Active Datasets Sort: Created Columns + New run

Run Name	Created	Dataset	Duration	Source	Models	Metrics
burly-shark-307	1 hour ago	dataset (e2b8386b) Mlflow_evaluation...	1.5min	mlflow-...	main.default.xgboo... +1	best_rmse 5.6722942... mflow_eva 400
industrious-du...	1 hour ago	-	471ms	mlflow-...	-	- - - -
bustling-sow-6...	1 hour ago	-	1.9s	mlflow-...	-	- - - -
fortunate-cod-...	1 hour ago	-	1.3s	mlflow-...	-	- - - -

Download 52 runs
Download all runs
Hide charts with no data
Auto-refresh

2. To download a file in CSV format containing all runs shown (up to a maximum of 100), select **Download <n> runs**. MLflow creates and downloads a file with one run per row, containing the following fields for each run:

Start Time, Duration, Run ID, Name, Source Type, Source Name, User, Status, <parameter1>, <parameter2>, ..., <metric1>, <metric2>, ...

3. If you want to download more than 100 runs or want to download runs programmatically, select **Download all runs**. A dialog opens showing a code snippet that you can copy or open in a notebook. After you run this code in a notebook cell, select **Download all rows** from the cell output.

Delete runs

You can delete runs from the experiment details page following these steps:

1. In the experiment, select one or more runs by clicking in the checkbox to the left of the run.
2. Click **Delete**.
3. If the run is a parent run, decide whether you also want to delete descendant runs. This option is selected by default.
4. Click **Delete** to confirm. Deleted runs are saved for 30 days. To display deleted runs, select **Deleted** in the State field.

Bulk delete runs based on the creation time

You can use Python to bulk delete runs of an experiment that were created prior to or at a UNIX timestamp. Using Databricks Runtime 14.1 or later, you can call the `mlflow.delete_runs` API to delete runs and return the number of runs deleted.

The following are the `mlflow.delete_runs` parameters:

- `experiment_id`: The ID of the experiment containing the runs to delete.
- `max_timestamp_millis`: The maximum creation timestamp in milliseconds since the UNIX epoch for deleting runs. Only runs created prior to or at this timestamp are deleted.
- `max_runs`: Optional. A positive integer that indicates the maximum number of runs to delete. The maximum allowed value for `max_runs` is 10000. If not specified, `max_runs` defaults to 10000.

Python

```
import mlflow

# Replace <experiment_id>, <max_timestamp_ms>, and <max_runs> with your
# values.
runs_deleted = mlflow.delete_runs(
    experiment_id=<experiment_id>,
    max_timestamp_millis=<max_timestamp_ms>,
    max_runs=<max_runs>
)

# Example:
runs_deleted = mlflow.delete_runs(
    experiment_id="4183847697906956",
    max_timestamp_millis=1711990504000,
```

```
max_runs=10
)
```

Using Databricks Runtime 13.3 LTS or earlier, you can run the following client code in a Databricks Notebook.

Python

```
from typing import Optional

def delete_runs(experiment_id: str,
                max_timestamp_millis: int,
                max_runs: Optional[int] = None) -> int:
    """
    Bulk delete runs in an experiment that were created prior to or at
    the specified timestamp.
    Deletes at most max_runs per request.

    :param experiment_id: The ID of the experiment containing the runs to
    delete.
    :param max_timestamp_millis: The maximum creation timestamp in
    milliseconds
                                since the UNIX epoch for deleting runs.
    Only runs
                                created prior to or at this timestamp
    are deleted.
    :param max_runs: Optional. A positive integer indicating the maximum
    number
                                of runs to delete. The maximum allowed value for
    max_runs
                                is 10000. If not specified, max_runs defaults to
    10000.
    :return: The number of runs deleted.
    """
    from mlflow.utils.databricks_utils import get_databricks_host_creds
    from mlflow.utils.request_utils import augmented_raise_for_status
    from mlflow.utils.rest_utils import http_request

    json_body = {"experiment_id": experiment_id, "max_timestamp_millis":
    max_timestamp_millis}
    if max_runs is not None:
        json_body["max_runs"] = max_runs
    response = http_request(
        host_creds=get_databricks_host_creds(),
        endpoint="/api/2.0/mlflow/databricks/runs/delete-runs",
        method="POST",
        json=json_body,
```

 Ask Genie

```
)  
augmented_raise_for_status(response)  
return response.json()["runs_deleted"]
```

See the Databricks Experiments API documentation for parameters and return value specifications for [deleting runs based on creation time](#).

Restore runs

You can restore previously deleted runs from the UI as follows:

1. On the **Experiment** page, in the **State** field, select **Deleted** to display deleted runs.
2. Select one or more runs by clicking in the checkbox to the left of the run.
3. Click **Restore**.
4. Click **Restore** to confirm. The restored runs now appear when you select **Active** in the State field.

Bulk restore runs based on the deletion time

You can also use Python to bulk restore runs of an experiment that were deleted at or after a UNIX timestamp. Using Databricks Runtime 14.1 or later, you can call the `mlflow.restore_runs` API to restore runs and return the number of restored runs.

The following are the `mlflow.restore_runs` parameters:

- `experiment_id`: The ID of the experiment containing the runs to restore.
- `min_timestamp_millis`: The minimum deletion timestamp in milliseconds since the UNIX epoch for restoring runs. Only runs deleted at or after this timestamp are restored.
- `max_runs`: Optional. A positive integer that indicates the maximum number of runs to restore. The maximum allowed value for `max_runs` is 10000. If not specified, `max_runs` defaults to 10000.

Python

```

import mlflow

# Replace <experiment_id>, <min_timestamp_ms>, and <max_runs> with your
values.
runs_restored = mlflow.restore_runs(
    experiment_id=<experiment_id>,
    min_timestamp_millis=<min_timestamp_ms>,
    max_runs=<max_runs>
)

# Example:
runs_restored = mlflow.restore_runs(
    experiment_id="4183847697906956",
    min_timestamp_millis=1711990504000,
    max_runs=10
)

```

Using Databricks Runtime 13.3 LTS or earlier, you can run the following client code in a Databricks Notebook.

Python

```

from typing import Optional

def restore_runs(experiment_id: str,
                 min_timestamp_millis: int,
                 max_runs: Optional[int] = None) -> int:
    """
    Bulk restore runs in an experiment that were deleted at or after the
    specified timestamp.
    Restores at most max_runs per request.

    :param experiment_id: The ID of the experiment containing the runs to
    restore.
    :param min_timestamp_millis: The minimum deletion timestamp in
    milliseconds
                                since the UNIX epoch for restoring runs.
    Only runs
                                deleted at or after this timestamp are
    restored.
    :param max_runs: Optional. A positive integer indicating the maximum
    number
                                of runs to restore. The maximum allowed value for
    max_runs
                                is 10000. If not specified, max_runs defaults to
    10000.
    :return: The number of runs restored.
    """

```

 Ask Genie

```

from mlflow.utils.databricks_utils import get_databricks_host_creds
from mlflow.utils.request_utils import augmented_raise_for_status
from mlflow.utils.rest_utils import http_request
json_body = {"experiment_id": experiment_id, "min_timestamp_millis":
min_timestamp_millis}
if max_runs is not None:
    json_body["max_runs"] = max_runs
response = http_request(
    host_creds=get_databricks_host_creds(),
    endpoint="/api/2.0/mlflow/databricks/runs/restore-runs",
    method="POST",
    json=json_body,
)
augmented_raise_for_status(response)
return response.json()["runs_restored"]

```

See the Databricks Experiments API documentation for parameters and return value specifications for [restoring runs based on deletion time](#).

Compare runs

You can compare runs from a single experiment or from multiple experiments. The **Comparing Runs** page presents information about the selected runs in tabular format. You can also create visualizations of run results and tables of run information, run parameters, and metrics. See [Compare MLflow runs and models using graphs and charts](#).

The **Parameters** and **Metrics** tables display the run parameters and metrics from all selected runs. The columns in these tables are identified by the **Run details** table immediately above. For simplicity, you can hide parameters and metrics that are identical in all selected runs by toggling ☐ **Show diff only** .

Experiments > /Users/ > /mlflow_python_qs > Comparing 4 Runs from 1 Experiment

Comparing 4 Runs from 1 Experiment

▸ Visualizations

▼ Run details

Run ID:	ff86fd88c7c54705aa1cbd339...	06d989ef699146269a511d6c...	e15521b2e5a94105a88ac2c0...	e6698e47a1674b079f...
Run Name:				
Start Time:	2021-12-21 11:42:21	2021-12-08 17:44:12	2021-12-08 17:22:04	2021-11-15 14:45:06

▼ Parameters

Show diff only

max_depth	6	4	8	4
max_features	3	4	3	3
n_estimators	100	250	100	100

▼ Metrics

Show diff only

training_mae	31.73	0.028	0.02	38.52
training_mse	1420.1	0.008	0.004	2136.2

Compare runs from a single experiment

1. On the [experiment details page](#), select two or more runs by clicking in the checkbox to the left of the run, or select all runs by checking the box at the top of the column.
2. Click **Compare**. The Comparing **<N>** Runs screen appears.

Compare runs from multiple experiments

1. On the [experiments page](#), select the experiments you want to compare by clicking in the box at the left of the experiment name.
2. Click **Compare (n)** (n is the number of experiments you selected). A screen appears showing all of the runs from the experiments you selected.
3. Select two or more runs by clicking in the checkbox to the left of the run, or select all runs by checking the box at the top of the column.
4. Click **Compare**. The Comparing **<N>** Runs screen appears.

Compare runs using system tables

MLflow metadata for experiments and runs is also available in system tables, where you can leverage [Databricks SQL](#) and all the lakehouse tooling Databricks offers to analyze your experiment data. For more details, see [MLflow system tables reference](#).

Copy runs between workspaces

To import or export MLflow runs to or from your Databricks workspace, you can use the community-driven open source project [MLflow Export-Import](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Compare MLflow runs and models using graphs and charts

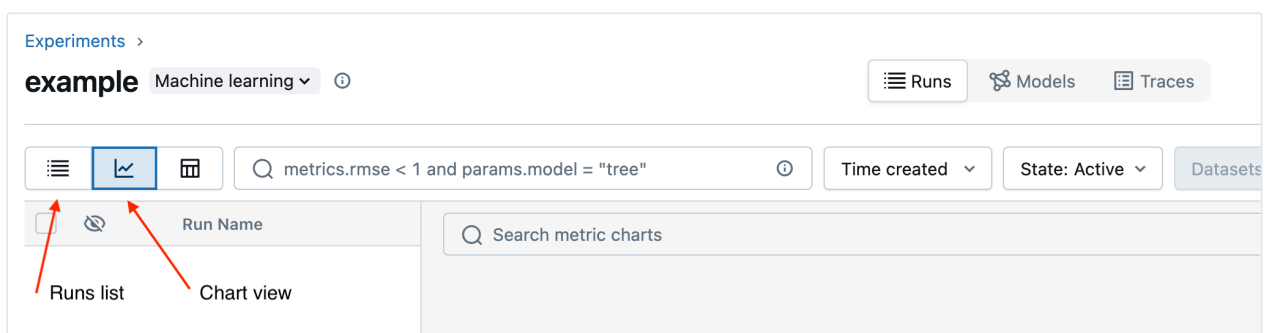
Last updated on **Feb 25, 2026**

Compare MLflow runs and models using graphs and charts

This page describes how to use the visualizations that are available for comparing runs in the MLflow UI. The chart view page shows a collection of charts comparing the runs of an experiment. You can customize this page by selecting runs to include, modifying charts, and creating new charts. With [MLflow 3](#), all of these features are available for models from the **Models** tab as well. For more details, see [Track and compare models using MLflow Logged Models](#).

MLflow metadata for experiments and runs is also available in system tables, where you can leverage [Databricks SQL](#) and all the lakehouse tooling Databricks offers to visualize your experiment data. See [MLflow system tables reference](#) for further details.

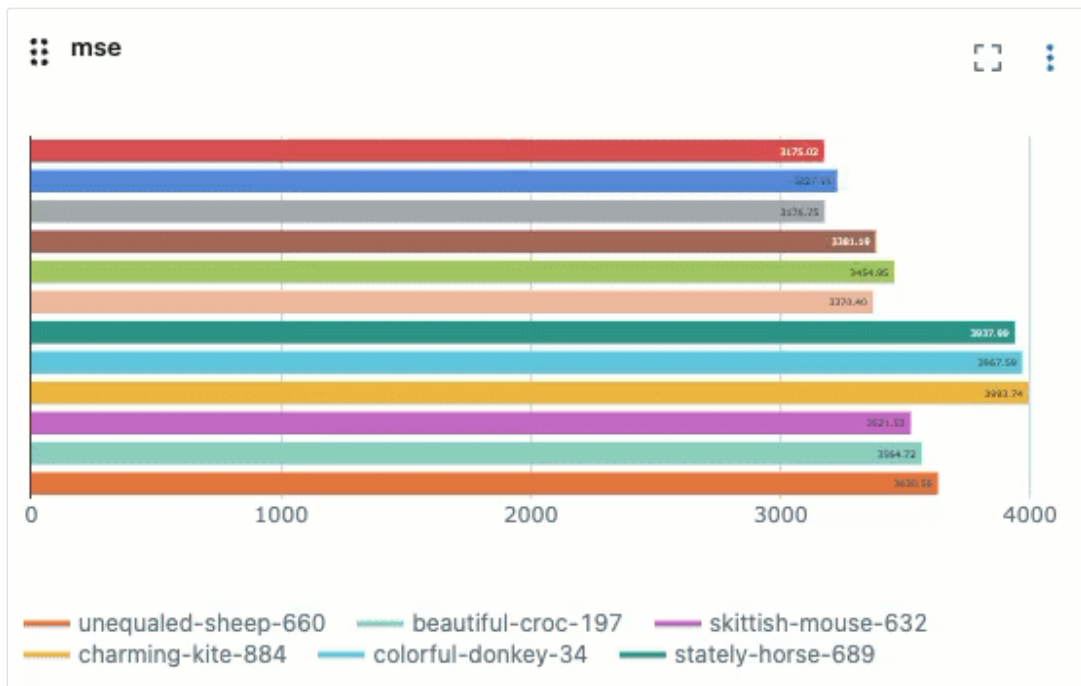
To display the chart view page, click the **Chart view** icon on the experiment details page.



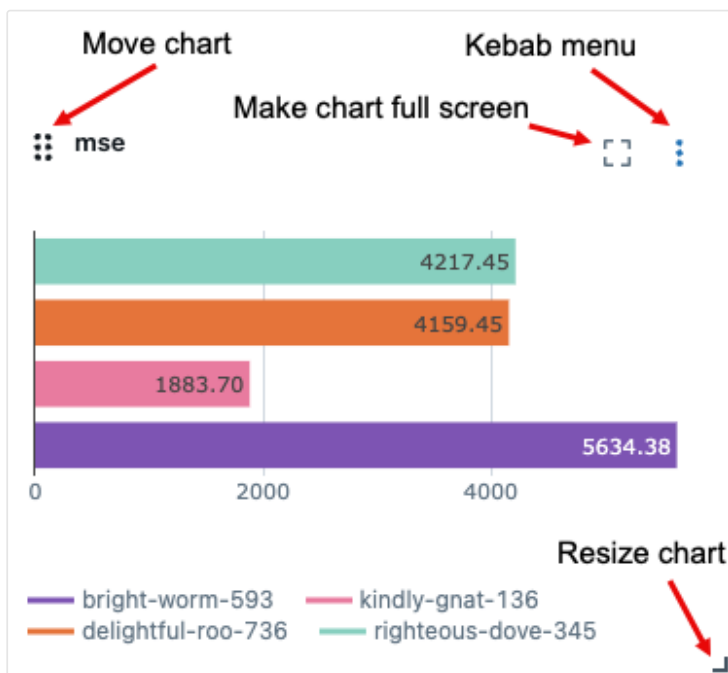
For information about the runs list page, see [View training results with MLflow runs](#). To display runs from multiple experiments, see [Compare runs from multiple experiments](#).

Chart overview

By default, charts on this page show the most recent 10 runs. As you roll your cursor over the lines on a chart, details for that run appear.



You can move or resize a chart, or enlarge it to full screen. A kebab menu at the upper-right of the chart lets you edit, delete, or download the chart.



Select runs to display

To select the number of runs to display, click  at the top of the list of runs.  Ask Genie



Runs that are shown on the charts are indicated by  and a colored dot. Runs not shown on the charts are indicated by  and a grayed-out dot.

☐		Run Name
☐		  xgboost_models
☐		 victorious-ray-243
☐		 amusing-mink-577
☐		 valuable-auk-239
☐		 gentle-seal-518

Manage runs

To delete, compare, or add or remove tags from a run, check the box next to the left of the run(s). When one or more runs is checked, the **Delete**, **Compare**, and **Add tags** buttons appear.

Experiments >
MLflow 3.0 Traditional ML Exam...
Machine learning
Runs
Models
Traces

Rename
Delete
Compare
Add tags

☒	Run Name	Created	Dataset	Duration	Source
☒	 skillful-snake-643	 6 days ago	 test (cc28e2f0)	2.2s	 mlflow3...
☒	 righteous-eel-476	 6 days ago	 train (d95ce242)	9.3s	 mlflow3...

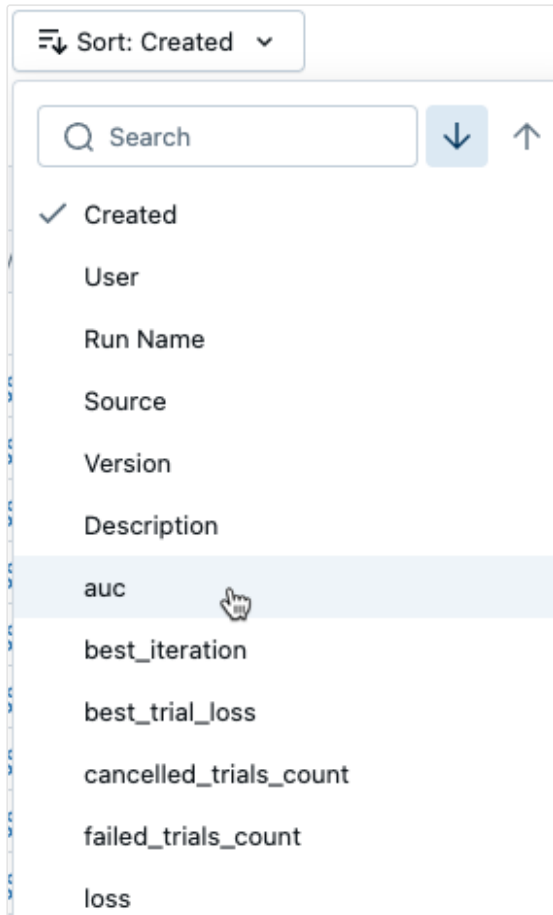
For details about the comparing runs page, see [Compare runs](#).

Filter runs

Use the search field to the right of the **Chart view** icon to filter runs based on parameter or metric values or by tag. For details, see [Filter runs](#).

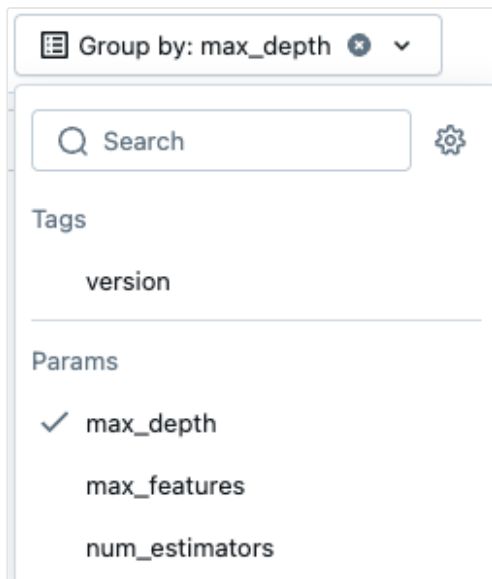
Sort runs

To change the sort order of runs shown in the charts, select the parameter to sort by from the **Sort** dropdown menu.



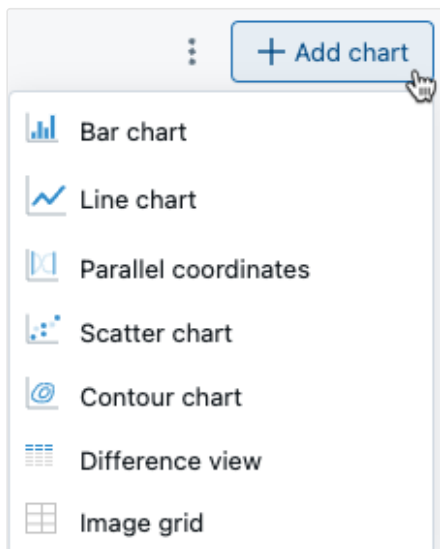
Group runs

To group runs by parameter value, select one or more parameters from the **Group by** dropdown menu.



Create new visualizations

To add a chart, click **Add chart**, and select the type of chart to add from the dropdown menu.



Parallel coordinates chart

A parallel coordinates plot is useful in understanding the effect of parameter settings on model performance and investigating relationships between parameters and metrics. To create a parallel coordinates plot, select **Parallel coordinates** from the menu. In the dialog, select the parameters and metrics to investigate.

Add new chart

Chart type

 Parallel coordinates ▼

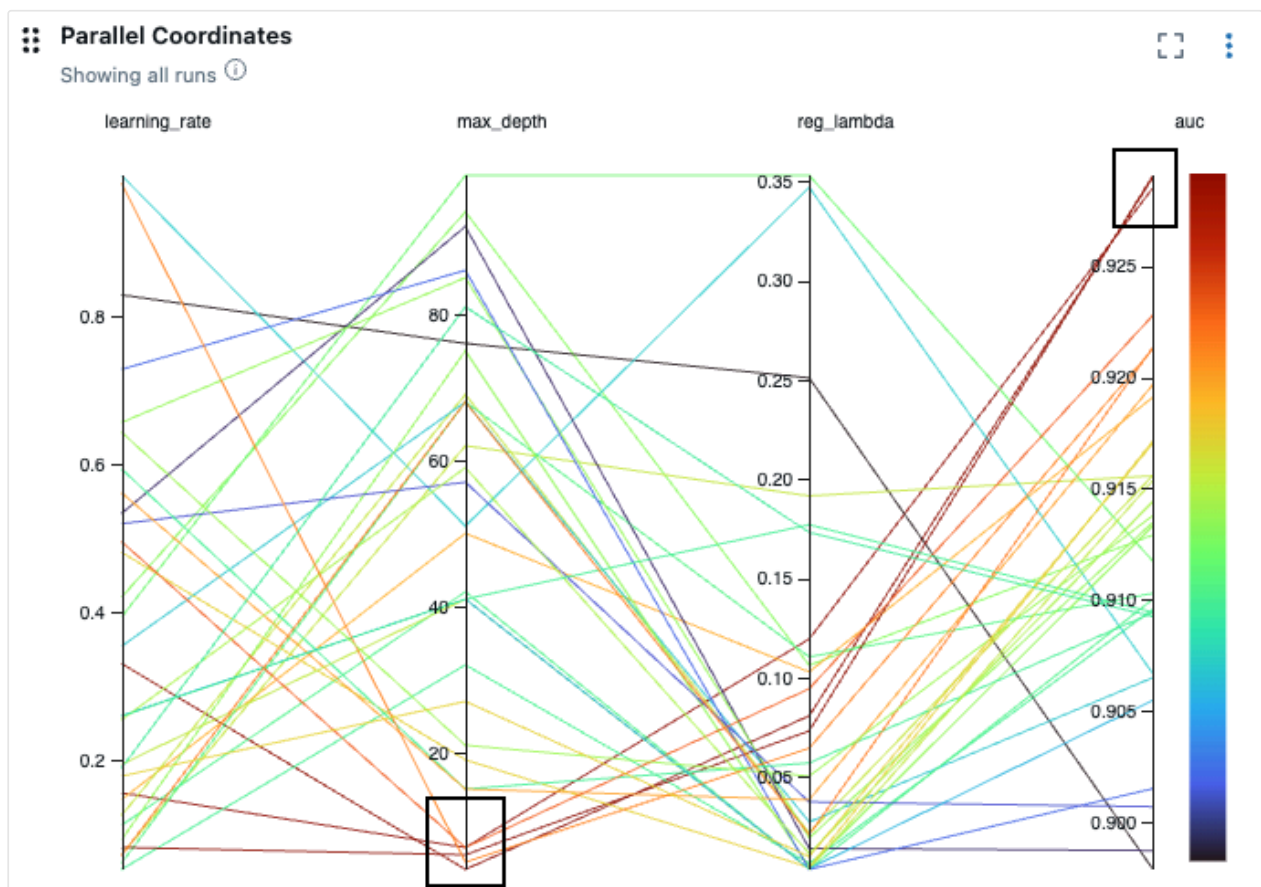
Params

`max_depth` × `learning_rate` × ▼
`reg_lambda` ×

Metrics

`auc` × ▼

In this example, the runs highlighted in the black boxes suggest that lower values for `max_depth` result in higher values for the metric `auc`.



Is this page
as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Feb 25, 2026**

Build dashboards with MLflow metadata in system tables

ⓘ PREVIEW

This feature is in [Public Preview](#).

Using MLflow metadata in [system tables](#), you can build [dashboards](#) to analyze your MLflow experiments and runs from the entire workspace. Using the existing MLflow UI and REST APIs for these tasks would require extensive, time-consuming iteration.

Dashboard for single run details

To start visualizing your MLflow data, download [this example dashboard](#) as a JSON file and [import it into your workspace](#). This dashboard contains a skeleton of data to replicate what is shown on the run details page in the MLflow UI.

For a given experiment ID, run ID, and metric name, it displays run details along with tags, parameters, and a metric graph. You can obtain the experiment ID and run ID from the run details page, both from the UI and in the URL itself:

```
https://<workspace>.databricks.com/ml/experiments/<experiment_id>/runs/<run_id>.
```


Experiments > /Users/ > /Notebook Experiment > Runs >

enchancing-panda-312 [Send feedback](#)

Reproduce Run

Register model

OverviewModel metricsSystem metricsTracesEvaluation resultsArtifacts

Description

No description

Metrics (16)

Search metrics

Metric	Latest	Min	Max	Models
System metrics (14)				
system/disk_available_megabytes	209155.7	209155.7	209156.4	-
system/disk_usage_percentage	18.2	18.2	18.2	-
system/gpu_0_memory_usage_percenta...	99.1	99.1	99.1	-
system/gpu_0_power_usage_watts	25.6	25.5	26.6	-

Parameters (1)

Search parameters

Parameter	Value
gpu_0_name	Tesla T4

About this run

Created at

Aug 17, 2025, 10:50 AM

Created by

Experiment ID

4335783192448869

Status

Finished

Run ID

309030b17eb84e1bb23dce0dd2e9e64

Duration

58.6s

Source

Notebook Experiment

Datasets

[train \(1f1c13b5\)](#)

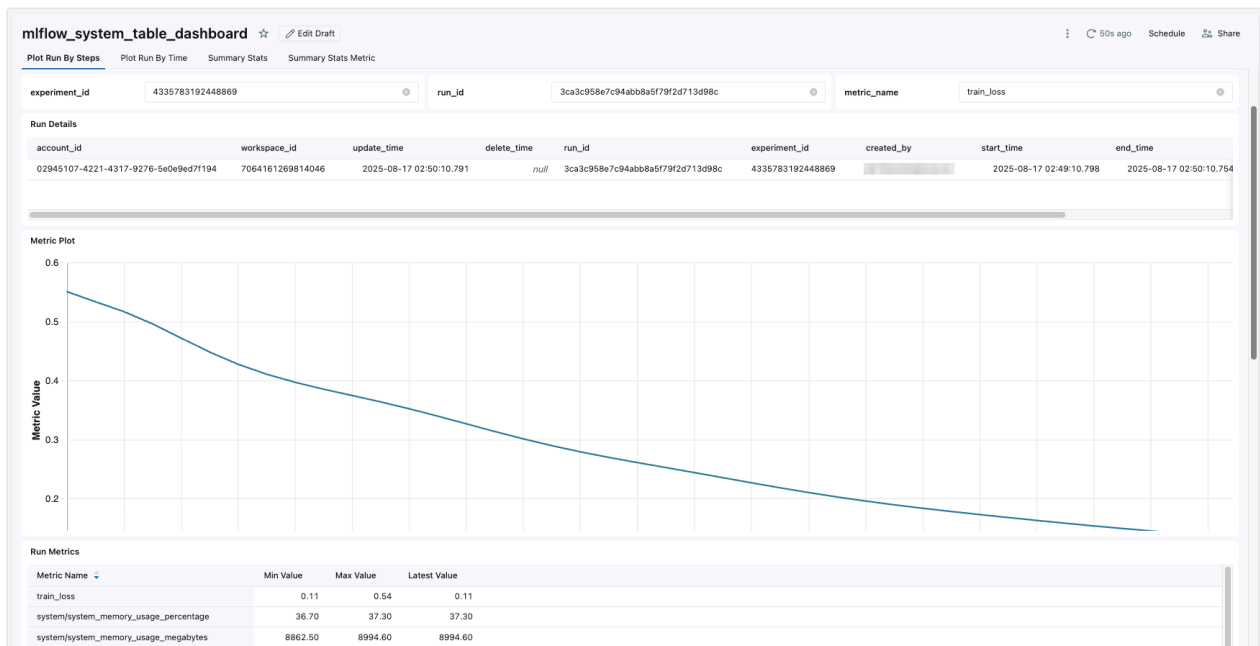
Tags

[Add tags](#)

Registered models

None

If you navigate to the dashboard panel from the left navigation menu, you can import the dashboard from a JSON file definition [here](#). From there, you can use the input boxes on the top to filter for the relevant run and experiment within your workspace to plot. Feel free to explore the queries and change the plots to meet your needs.



mlflow_system_table_dashboard ☆ View Published

Serverless Starter En... Publish Share

Data Plot Run By Steps Plot Run By Time Summary Stats Summary Stats Metric +

Datasets

- runs_latest_filtered
- explode_tags_filtered
- explode_params_filtered
- explode_metrics_filtered
- single_metric_values
- summary_stats_experiment_metric
- summary_stats_experiment

Create another dataset

- + Create from SQL
- + Add data source
- Upload file

Run 8 minutes ago system mlflow

```
1 SELECT
2   metric_name,
3   metric_value,
4   metric_step,
5   metric_time
6 FROM system.mlflow.run_metrics_history
7 WHERE experiment_id LIKE :experiment_id
8 AND run_id LIKE trim(:run_id)
9 AND metric_name LIKE trim(:metric_name)
10 LIMIT 1000000
```

experiment_id 4335783192448868 run_id 4ca4b98741ce4367ae3 Metric Name train_loss + Add parameter

Result Table Schema Custom Calculation

	metric_name	metric_value	metric_step	metric_time
1	train_loss	0.4107516407966614	7	2025-08-17T02:32:39.048+00:00
2	train_loss	0.24417391419410706	22	2025-08-17T02:32:54.873+00:00
3	train_loss	0.18383340537548065	30	2025-08-17T02:32:57.146+00:00
4	train_loss	0.35219353437423706	12	2025-08-17T02:32:45.609+00:00
5	train_loss	0.3853282928466797	9	2025-08-17T02:32:39.607+00:00
6	train_loss	0.1731252670288086	32	2025-08-17T02:33:03.078+00:00
7	train_loss	0.2269265353679657	24	2025-08-17T02:32:55.449+00:00
8	train_loss	0.1382431834936142	40	2025-08-17T02:33:05.416+00:00
9	train_loss	0.1119222342967987	50	2025-08-17T02:33:14.594+00:00

> See performance Statement 1/1 Optimize

Dashboard to monitor average GPU utilization across experiments

On the fourth tab from the above dashboard, you can input a metric name to get summary statistics across all experiments with that metric within a given time window. This information can be useful for monitoring [system metrics](#) recorded by MLflow across your workspace to monitor for inefficient CPU, memory, or GPU utilization.

mlflow_system_table_dashboard ☆ Edit Draft

40s ago Schedule Share

Plot Run By Steps Plot Run By Time Summary Stats Summary Stats Metric

metric_name system/gpu_0_utilization_percentage start_time 14 days ago (Aug 03, 2025 00:00:00)

experiment_id	success_ratio	count_runs	runs_with_metric	avg_metric_value	max_metric_value	min_metric_value
3340239623640311	null	81	81	3.90	96.70	0.00
4335783192473836	1.00	22	22	11.60	55.00	0.00
4335783192473840	1.00	12	12	20.50	83.00	0.00
4335783192448869	1.00	10	5	3.70	10.00	0.00
4335783192448868	1.00	8	8	4.30	20.00	0.00

In the example, we can see several experiments with an average GPU utilization of less than 10% which we may want to investigate.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback